# COMSATS University Islamabad,
# Sahiwal Campus

# LuxeBook

**A Project Presented To**

**COMSATS University Islamabad,**
**Sahiwal Campus**

**In partial fulfillment**

**of the requirement for the degree of**

## Bachelor of Science in Computer Science (2022-2026)

## By


| | |
|---|---|
| **Ijtaba Zahra** | **CIIT/FA22-BCS-256/SWL** |
| **Ishrat Jamil** | **CIIT/FA22-BCS-166/SWL** |

# Declaration

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

**Ijtaba Zahra** **Ishrat Jamil**

# Executive Summary

The Salon Management System is a comprehensive web-based platform designed to digitize and streamline the daily operations of modern salons and beauty parlors. The platform offers a centralized space where salon owners can manage appointments, staff schedules, services, payments, and customer relationships efficiently. Unlike traditional manual systems that rely on paper registers or scattered Excel sheets, this system provides a complete digital solution tailored specifically for small to medium-sized salons.

The system helps salon owners automate their booking processes, track customer history, manage staff commissions, and view real-time business analytics in a structured, professional, and visually appealing dashboard format. Unlike generic scheduling apps, this system focuses specifically on salon-specific workflows and ensures owners receive complete visibility and control over their business operations.

To support salon owners who struggle with manual record-keeping and missed appointments, the system integrates an automated appointment reminder module. This feature sends email notifications to customers before their scheduled appointments, reduces no-shows, identifies booking patterns, and provides operational insights to help owners improve their business performance and customer retention.

The platform includes a comprehensive booking engine, allowing customers to browse services, select preferred staff members, choose available time slots, and make secure online payments via Stripe integration. This ensures that even salons without a digital presence can offer a modern, convenient booking experience to their customers.

For customers, the system offers a straightforward and intuitive interface where they can view service catalogs, check staff availability, book appointments, make payments, view booking history, and leave reviews. The system does not use complex automated ranking or biased recommendation systems, ensuring transparent, customer-centered service discovery.

Built using modern technologies such as Node.js, Express.js, React.js, PostgreSQL, Prisma ORM, Redis, Bull Queue, and Stripe, the system ensures secure authentication and smooth data processing. The platform includes automated background jobs for sending reminders, cleaning up old bookings, and expiring subscriptions. The project follows an Agile development model, allowing continuous improvements through real-user feedback from salon owners, customers, and staff members. Comprehensive testing including unit, integration, system, and functional testing ensures reliability, accuracy, and seamless functionality across all features.

# Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **“Ms. Asma Iqbal”** and our co-supervisor **“Dr.Yawar ”.** Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty and hard work.

# Abbreviations

| | |
|---|---|
| **UT** | Unit Testing |
| **FT** | Functional Testing |
| **IT** | Integration Testing |
| **CUI** | Comsats University Islamabad |
| **UC** | Use Case |
| **API** | Application Programming Interface |
| **BSc** | Bachelor of Science |
| **COMSATS** | COMSATS University Islamabad |
| **CSS3** | Cascading Style Sheets 3 |
| **HTML5** | HyperText Markup Language 5 |
| **IT** | Integration Testing |
| **PDF** | Portable Document Format |
| **PostgreSQL** | PostgreSQL (relational database) |
| **RBAC** | Role-Based Access Control |
| **UI** | User Interface |
| **UI/UX** | User Interface/User Experience |
| **UC** | Use Case |
| **UT** | Unit Testing |

# Table of Contents

# List of Figures

# List of Tables

# Chapter 1
# Introduction

# 1. Introduction

The Salon Management System is a modern, web-based application designed to empower salon owners and customers as they navigate the challenges of appointment booking, staff management, and payment processing. In an increasingly competitive beauty and wellness industry, many salon owners struggle with manual record-keeping, double-bookings, lack of customer history, and difficulty presenting their services in a professional digital format. The Salon Management System addresses these challenges by providing a unified space for online booking, staff scheduling, payment processing.

Unlike traditional paper registers or scattered Excel sheets, this system enables salon owners to build a structured, visually appealing, and comprehensive digital platform that highlights their services, staff expertise, and business operations. The platform also integrates an automated appointment reminder module, allowing customers to receive notifications before their scheduled visits. This module tracks booking patterns, reduces no-shows, and offers operational insights to help salon owners improve their business performance.

Developed using Node.js, Express.js, React.js, PostgreSQL, Prisma ORM, Redis, Bull Queue, and Stripe, following Agile methodology, the system ensures a secure environment accessible to all users. While the platform uses automated queues and cron jobs for reminders and cleanup, it avoids complex ranking systems, ensuring fair and transparent service.

For customers, the system provides a portal to browse services, view staff availability, book appointments, make secure payments, and leave reviews. Customers can browse services, view staff availability, book appointments, make secure payments, and leave reviews.

For providers, the system offers a dashboard to manage services, staff schedules, view analytics, track expenses, and manage subscriptions. Role-based access control ensures owners have full control over their operations.

Security is ensured through JWT authentication with HTTP-only cookies, password hashing, Prisma ORM, and thorough testing including unit, integration, functional, and system testing.

This project, developed at COMSATS University Islamabad, Sahiwal Campus, is deployment-ready and can evolve through real-user feedback. The Salon Management System aims to strengthen salon visibility, enhance customer experience, and build a direct bridge between salon services and customer needs.

## 1.1 Brief Overview

The Salon Management System project aims to develop a web-based salon management platform designed exclusively for salon owners, staff, and customers to enhance how they manage, present, and grow their salon operations. The system will be developed using HTML5, CSS3, JavaScript, Bootstrap 5, and React.js for the front end; Node.js 20.x, Express.js 4.x, and Prisma ORM for the backend; and SQLite for development with PostgreSQL in production to ensure a secure, scalable, and high-performance experience. The project adopts agile software development methodology, enabling iterative and structured development.

Salon Management System is a unique and focused platform designed to provide users with an intelligent space where they can manage appointments, track customer history, and receive automated insights. While many professional tools exist, Salon Management System stands out due to its specialized focus on salon operations, making it a dedicated hub for salon owners and their customers. It addresses challenges like scattered records, limited visibility, and weak customer management by enabling users to organize their salon data, leverage automated reminders, and access business insights ensuring they stay informed and prepared for opportunities that align with their business growth.

## 1.2 Relevance to Course Modules

The development of Customized Salon Management System directly applies and integrates various concepts and skills acquired throughout our Computer Science course.

**i. Technical Skills:**

The project involves the application of multiple programming languages, frameworks, and tools learned during the course, including HTML5, CSS3, JavaScript, Bootstrap 5, React.js, and backend development using Node.js and Express.js. These skills were essential for building the salon management system and implementing core functionalities such as user authentication, appointment booking, staff management, and payment processing. The principles of software engineering such as modular design, MVC architecture, and secure coding practices were applied to ensure system reliability, scalability, and maintainability.

**ii. Software Engineering Principles:**

The project adheres to software engineering principles covering requirement analysis, system design, software architecture, testing, and agile development. Each phase of the Salon Management System development follows structured practices to ensure scalability, maintainability, usability, and security. The system integrates concepts from web development, distributed systems, version control, and cloud deployment making it a comprehensive computer science application.

**iii. Database Design:**

A strong understanding of database management was essential in designing the SQLite (development) and PostgreSQL (production) databases for storing user profiles, salon services, staff details, appointments, payments, and subscription records. The platform makes use of relational mapping, indexing, and PostgreSQL full-text search to ensure fast and efficient retrieval of salon data across large datasets.

**iv. System Architecture:**

The project required an understanding of modern web system architecture. Concepts such as Express.js MVC architecture, REST API design, and modular structure were applied to build a structured and scalable system. The backend follows a layered architecture with routes, controllers, middleware, and services. The frontend uses React.js component-based architecture for reusable UI elements.

**v. UI/UX Design:**

A user-friendly interface is essential for the success of a salon management platform. Concepts from human-computer interaction and UI/UX design courses guided the creation of clean, responsive layouts using Bootstrap 5 and CSS3. These principles helped build an intuitive and visually consistent interface that customers, salon owners, and admins can use comfortably across devices.

**vi. Project Management:**

Project management concepts played a critical role in planning, executing, and controlling the development phases of Salon Management System. Agile methodology supported iterative development, sprint-based progress, and continuous improvement. This ensured that features such as booking modules, provider dashboards, and payment integration were delivered in a timely and structured manner

**vii. Networking and AI Integration:**

Salon Management System includes features such as appointment booking, payment processing, and real-time availability checking, which rely on concepts from API communication and networking principles. Additionally, real-time updates such as booking status changes and staff schedule management depend on efficient API communication and backend networking principles. Understanding HTTP protocols, REST standards, and asynchronous operations ensures seamless interaction across the platform.

## 1.3 Project Background

The Salon Management System was conceptualized in September 2025 at COMSATS University Islamabad, Sahiwal Campus, under the supervision of Mr. Ahmad Shaf, as part of the Bachelor of Science in Computer Science (2022–2026) final year project. The idea originated from the real difficulties faced by salon owners and customers during their daily business operations. Many salon owners shared that they struggled with manual appointment booking, lost customer records, and difficulties tracking staff schedules and revenue. Customers similarly expressed that calling salons for appointments was inconvenient, especially during busy hours, and there was no way to check staff availability or book services online.

A survey conducted with thirty participants, including salon owners, staff members, and regular customers, revealed widespread issues in salon management. Salon owners reported double-bookings, no-shows, difficulty tracking customer history, and confusion about how to manage staff commissions and business analytics. Many highlighted that traditional methods such as paper registers or WhatsApp messages failed to provide a structured and reliable system for managing daily operations. Customers, on the other hand, noted that they often had to wait on phone calls or visit salons in person to book appointments, and there was no digital record of their service history.

The Salon Management System was designed to address these concerns by providing a dedicated space where customers can book appointments online, view staff availability, and make secure payments. The system also includes an automated reminder module that sends email notifications to customers before their scheduled appointments, reducing no-shows and improving customer retention. For salon owners, the platform offers a comprehensive dashboard to manage services, staff schedules, track revenue, and view business analytics. Recruiters are not part of this system as it is focused on salon-customer interactions.

Developed using Node.js, Express.js, React.js, PostgreSQL, Prisma ORM, Redis, Bull Queue, and Stripe Payment Gateway, following Agile methodology, the project emphasizes secure authentication, efficient data management, and an intuitive user experience suitable for users from all technical backgrounds. Over a six-month development timeline, the platform progressed through requirement analysis, design, implementation, testing, and deployment phases, ensuring reliability and usability. The Salon Management System aligns with the university's mission to encourage innovation and practical skill development, offering a scalable solution that enhances salon visibility, improves customer experience, and facilitates meaningful engagement between salons and their customers.

.

## 1.4 Literature Review

Table 1.1 states that Salon Management System platform builds upon foundational research in web-based appointment systems, staff management, payment integration, and business analytics. This review synthesizes key studies and frameworks that validate the need for an integrated, automated salon management system as given in table 1.1.

**Table 1.1** Literature Review

| Application Name | Weakness | Proposed Project Solution (Customized Academic Portfolio) |
|---|---|---|
| **CloudPOS** | High subscription cost for small salons, feature overload including batch expiry tracking and chair utilization logic, designed for multi-branch operations, requires technical training to use effectively | Simple interface with only essential features, designed specifically for single-location small salons, no training required, affordable PKR pricing |
| **MyPOS** | POS-focused with no customer online booking portal, requires hardware terminal for payments, no customer dashboard for booking history, no automated reminders | Pure software solution with no hardware required, 24/7 customer online booking portal, complete customer dashboard with booking history, automated email reminders |
| **Websol POS** | No online appointment booking system, customers | Self-service online booking 24/7, customer |

| | | |
|---|---|---|
| | must call or visit salon to book, no customer self-service portal, no automated reminders, POS-focused only | dashboard to view and manage appointments, automated email reminders, appointment-first design not POS-focused |
| **Manual Systems (Paper/Excel)** | No digital backup, no real-time sync, easy to lose records for Pakistani salons with no IT infrastructure | Digital database with backups, real-time sync, designed for salons with basic IT literacy |

## 1.5 Analysis from Literature Review

This section evaluates findings from the literature related to appointment booking systems, staff management, payment integration, and UI/UX best practices for salon management. The reviewed research emphasizes the importance of structured appointment scheduling, automated reminders, and platforms designed specifically for small to medium-sized salons. Studies highlight that salon owners benefit most from systems that offer real-time availability, customer history tracking, and simplified staff management. Research also underscores the limitations of manual registers and scattered WhatsApp messages, suggesting the need for centralized platforms that support salon operations without overwhelming technical complexity.

Unlike platforms relying on complex enterprise features or high subscription fees, the literature supports solutions that enhance operational efficiency, improve customer communication, and promote transparent owner-customer engagement. These insights justify our Salon Management System's focus on online booking, automated reminders, staff scheduling, and integrated payment processing, ensuring salon owners gain both efficiency and visibility as they manage their daily operations.

### 1.5.1 CloudPOS

CloudPOS is a popular salon and retail management platform widely used in the Pakistani market. However, it has several limitations for small, single-location salon owners. First, its subscription fee is expensive for basic features, and the platform requires significant upfront investment for small businesses. Second, CloudPOS is primarily focused on multi-branch retail operations and large salons, meaning its features include complex inventory management, batch expiry tracking, chair utilization logic, and FBR integration that small salons do not need. Third, the platform has a steep learning curve due to feature overload, requiring technical expertise and staff training to operate effectively. Our Salon Management System addresses these gaps by offering a lower cost solution with a simple interface designed specifically for small Pakistani salons. The platform includes only essential features that local salon owners actually need, such as online booking, staff management, and basic analytics, making it easy to use without technical expertise.

### 1.5.2 Mannual systems

Many small salons in Pakistan still rely on manual systems such as paper registers or Excel spreadsheets. These systems have critical weaknesses. Paper registers have no digital backup if the register is lost or damaged, all customer records and appointment data are gone forever. There is no real-time synchronization, meaning if two staff members maintain separate records, double-bookings occur frequently. For salons with no IT infrastructure, manual records are easy to lose or misplace. Our Salon Management System provides a digital database with automatic backups, ensuring no data loss. Real-time synchronization means all staff members see the same availability instantly. The system is designed specifically for salons with basic IT literacy, with simple interfaces and clear instructions.

### 1.5.3 Websol POS

Websol POS is a point-of-sale system adapted for salon billing in Pakistan. However, it has several limitations for modern salon operations. First, Websol POS lacks an online appointment booking system, meaning all appointments must be scheduled via phone or in-person visits. Second, there is no customer dashboard where clients can view their booking history, upcoming appointments, or payment records. Third, the system does not send automated reminders or confirmations, resulting in higher no-show rates and manual follow-up work for salon staff. Our Salon Management System addresses these gaps by offering a complete online booking experience with customer dashboards, booking history, automated email confirmations, and 24/7 appointment scheduling, transforming the salon from manual to fully digital operations.

### 1.5.4 SalonFlow

SalonFlow is a salon management platform targeting the Pakistani market. However, it has several limitations for small salon owners. First, publicly available technical documentation and detailed feature information is limited, making it difficult to evaluate platform capabilities before commitment. Second, the platform's technology stack and payment integration details are not clearly documented, creating uncertainty about reliability and security. Third, there is limited information about automated notification features, customer self-service portals, or staff management tools. Our Salon Management System addresses these gaps by providing complete transparency through detailed documentation, clearly defined features including Stripe payment integration, automated email reminders, customer dashboards, and staff management tools. The platform's modern technology stack (React.js, Node.js, PostgreSQL) is fully documented and publicly verifiable.

## 1.6 Methodology and Software Lifecycle

Salon Management System is a software project with diverse and evolving requirements, making it important to use a development methodology that supports flexibility, collaboration, and continuous refinement. Agile is an ideal fit because it allows the system to grow iteratively while adapting to user needs across customers, salon owners, and administrators.

### 1.6.1 Rationale Behind Selected Agile Model

Agile emphasizes adaptability, teamwork, and ongoing enhancement, making it well suited for a platform like Salon Management System that requires frequent updates and user-driven improvements. Since the system involves continuous interaction with users such as managing appointments, updating staff schedules, and improving payment integration, the Agile model ensures the platform evolves in sync with real-world salon business needs.

Agile is based on the following principles:

1. **Iterative development:** Core features such as user authentication, booking engine, payment integration, and admin dashboard are developed in cycles, with each iteration refining functionality.
2. **Incremental delivery:** Modules are released in small, usable increments, enabling early access to features like customer booking, provider dashboard, and appointment calendar.
3. **Continuous feedback:** Customers, salon owners, and administrators provide ongoing input regarding usability, feature relevance, and system reliability.
4. **Adaptive planning:** The development team refines plans based on feedback, evolving salon workflows, and new feature requests, ensuring the Salon Management System remains aligned with user expectations.

## 1.7 Summary

The Salon Management System, initiated in September 2025 at COMSATS University Islamabad, Sahiwal Campus, was developed to address the practical challenges faced by salon owners, staff members, and customers in managing and accessing salon services. Traditional methods such as paper registers, scattered WhatsApp messages, and manually maintained Excel sheets often fail to provide a structured, complete, and professional system for daily salon operations. Existing local platforms like CloudPOS, InstaCare, MyPOS, Websol POS, and SalonFlow either have complex feature sets designed for multi-branch operations, lack customer self-service portals, do not offer online booking for customers, or require technical expertise that many small salon owners do not possess. International platforms like Booksy, Vagaro, Fresha, and Square Appointments have high USD subscription fees, lack local payment integration, or are not available in Pakistan.

To overcome these limitations, the Salon Management System provides a centralized, user-friendly platform where customers can book appointments online 24/7, view staff availability, make secure payments via Stripe, and view their service history in a polished and professional manner. Unlike CloudPOS which focuses on complex inventory and multi-branch features, or MyPOS and Websol POS which are POS-focused without customer self-booking, our system prioritizes customer convenience with automated appointment reminders that send email notifications before scheduled visits, reducing no-shows and improving customer retention. For salon owners, the platform offers a dedicated dashboard that allows them to manage services, staff schedules, track revenue, view business analytics, and manage subscriptions without relying on complex or expensive software.

Developed using the Agile methodology, the Salon Management System emphasizes iterative development, continuous feedback, and adaptive planning to ensure that the platform meets real user needs. Through modern web technologies including Node.js, Express.js, React.js, PostgreSQL, Prisma ORM, Redis, Bull Queue, and Stripe, along with secure authentication and an accessible interface, the system supports smooth booking, reliable data handling, and an environment that enhances salon visibility and improves customer experience. The Salon Management System ultimately serves as a comprehensive digital space for appointment booking, staff management, payment processing, and meaningful salon-customer interaction, specifically designed for small, single-location salons in the Pakistani market.

# Chapter 2
# Problem Definition

# 2 Problem Definition

This chapter discusses the precise problem to be solved and the intended outcome.

## 2.1 Problem Statement

Salon owners and customers often struggle to manage and access salon services due to the absence of a centralized and dedicated platform. Traditional methods such as paper registers, scattered WhatsApp messages, manually updated Excel sheets, and generic scheduling apps fail to provide a salon-focused environment. As a result, salon owners face difficulties managing appointments, tracking customer history, coordinating staff schedules, and calculating revenue in a structured manner, leading to double-bookings, lost customer records, and reduced business efficiency. Customers also find it challenging to book appointments, check staff availability, or make online payments when salons rely on manual or disconnected systems.

To address these challenges, the Salon Management System is being developed as a unified online booking and business management platform. The system enables customers to book appointments online, view staff availability, make secure payments through Stripe, and view their service history. The platform also includes an automated appointment reminder feature, allowing customers to receive email notifications before their scheduled visits, reducing no-shows and improving customer retention. For salon owners, the system provides a dedicated dashboard where they can manage services, staff schedules, track revenue, view business analytics, manage expenses, and handle subscriptions.

## 2.2 Deliverable and Development Requirements

The Salon Management System project's deliverables and development requirements encompass a range of components essential for the successful implementation of a complete online booking and salon management platform. These components are detailed below:detailed below:

### 2.2.1 User Interface (UI) Design

Develop a clean, intuitive, and salon-focused interface that allows customers to easily browse services, book appointments, and make payments. Salon owners should be able to manage their business operations without confusion. The interface should be visually appealing and consistent across all modules, ensuring users can navigate without difficulty. Responsiveness across mobile devices, tablets, and desktops will be prioritized to enhance user engagement and satisfaction. Design elements will be kept simple and accessible, making the platform usable for salon owners and customers with basic technical literay.

### 2.2.2 User Registration and Profiles

Implement a secure registration and login system for customers and salon owners. Customers will have profiles containing their booking history, upcoming appointments, and payment records. Salon owners will have business profiles containing their salon information, service catalog, staff details, and business analytics.

### 2.2.3 Appointment Booking Engine

Provide a dynamic booking module where customers can browse available services, select preferred staff members, view real-time time slots, and confirm appointments online. The system will display only those time slots where both the selected staff member and the service duration are available. Customers can book appointments from any device without calling the salon. The system will prevent double-booking by checking availability in real-time before confirming any appointment. Once a booking is confirmed, the time slot is immediately blocked for other customers. Customers will receive an email confirmation with appointment details after successful booking. Salon owners can view all upcoming bookings in their dashboard calendar.

### 2.2.4 Payment Processing

Integrate Stripe payment gateway to enable secure online payments for appointments. Customers will enter their card details on a secure Stripe checkout page. The system will process payments and return success or failure status. Upon successful payment, the system will automatically update booking status from pending to confirmed. An invoice will be generated and sent to the customer via email. Failed payments will keep the booking pending and notify the customer to retry. Webhooks will be used to handle asynchronous payment confirmations from Stripe.

### 2.2.5 Staff Management Module

Develop a complete staff management system where salon owners can add, edit, or remove staff members. Each staff member will have a profile containing name, role, contact information, and commission rate. Owners can assign weekly working hours for each staff member, including break times. Staff schedules will be visible to customers during the booking process. Commission rates will be automatically calculated based on completed services. Time-off requests can be submitted by staff and approved by the owner. Staff performance metrics will be displayed in the analytics dashboard.

### 2.2.6 Dashboard for Customers and Sales owner

Provide separate dashboards for customers and salon owners based on their role in the system. Customer dashboard will show upcoming appointments, booking history, payment records, and options to cancel or reschedule bookings. Salon owner dashboard will display revenue analytics, total bookings, popular services, staff performance, and expense tracking. Both dashboards will use charts and graphs for clear data visualization. All dashboard data will update in real-time as new bookings and payments occur. Customers can save favorite services while owners can view daily, weekly, and monthly business reports.

### 2.2.7 Admin Panel

Develop an admin panel for platform supervision and management. Admins can view a complete list of all registered salon owners and customers. New provider registrations will appear with pending status requiring admin approval. Admins can approve or reject provider registrations based on business verification. All subscription plans and active subscriptions can be viewed and managed from the admin panel. System-wide analytics will show total users, total bookings, total revenue, and platform growth metrics. Admins can disable or delete user accounts in case of policy violations.

### 2.2.8 Notifications, Subscriptions and Expenses

Implement automated email notifications using Bull queue and Redis for booking confirmations, appointment reminders, and payment receipts. Provide subscription plans (Basic, Premium, Enterprise) for salon owners with Stripe payment processing and access control for premium features. Allow salon owners to track business expenses including rent, utilities, staff salaries, and product purchases with exportable reports. Enable customers to leave 5-star ratings and text reviews for completed services, visible on service and staff pages. Salon owners can respond to reviews publicly, and average ratings are displayed for each service and staff member. Subscription status, expense data, and review analytics are integrated into the owner dashboard for complete business oversight.indexing, and caching mechanisms for efficient data retrieval.

### 2.2.9 Security and Privacy

Ensure strong data protection using multiple security mechanisms throughout the platform. JWT tokens will be used for authentication and stored in HTTP-only cookies to prevent XSS attacks. Passwords will be hashed using bcrypt before storing in the database. Role-based access control will ensure customers can only access customer features and owners only owner features. All data transmission between frontend and backend will be encrypted using HTTPS. Session management will automatically log out inactive users after a defined time period. Regular security practices and dependency updates will be maintained to protect user data.

### 2.2.10 Documentation

Prepare comprehensive documentation for different audiences of the platform. User documentation for customers will include step-by-step booking instructions with screenshots. Owner documentation will cover dashboard navigation, service management, staff scheduling, and analytics. API documentation will be provided using tools like Swagger or Postman for developers. Installation guide will include requirements, environment setup, and deployment steps. Troubleshooting section will cover common issues and their solutions. All documentation will be maintained in PDF and web formats. Screenshots and diagrams will be included for visual clarity.

### 2.2.11 Testing and Quality Assurance

Conduct rigorous testing to ensure platform stability and error-free operation. Unit testing will verify individual functions such as authentication and booking creation. Functional testing will validate complete features against requirements. Integration testing will ensure modules like booking and payment work together correctly. Test cases will cover all modules including booking, payment, staff management, and analytics. Edge cases such as double-booking attempts and invalid payment methods will be tested. Bug tracking will be maintained throughout the development process. All test results will be documented with pass or fail status for each test case.

### 2.2.12 Regular Updates and Maintenance

Establish an ongoing maintenance and upgrade plan to introduce new features, improve performance, and ensure continued system security. This plan will include periodic testing, bug fixes, and dependency updates to maintain stability. User feedback from salon owners and customers will be collected to prioritize new features. Regular updates will ensure compatibility with evolving technologies like Stripe API updates and browser changes. Security patches will be applied promptly to protect user data and payment information.

## 2.3 Current System Limitations

Currently, there is no centralized platform that enables salon owners to professionally manage appointments, staff schedules, customer records, and payments in a structured and efficient way. Most salon owners rely on paper registers, scattered WhatsApp messages, manual Excel sheets, and generic scheduling apps, which are not tailored to salon-specific workflows. As a result, salon owners often struggle to manage double-bookings, track customer history, coordinate staff schedules, and calculate revenue accurately, leading to lost business and reduced customer satisfaction.

Customers also lack guided tools that help them check staff availability, book appointments online, or make secure digital payments. Without a dedicated system to book appointments or receive reminders, many customers experience long phone wait times, missed appointments, and difficulty maintaining service history with their preferred salons.

The following table 2.1 summarizes the limitations of existing salon management platforms and manual methods, emphasizing the need for an integrated, user-friendly solution like the Salon Management System, which focuses on online booking, staff scheduling, payment processing, and business analytics. These gaps highlight the absence of a unified system that streamlines salon operations while supporting business growth. The analysis further reinforces the importance of a platform that bridges salon services with customer needs as given in table 2.1

**Table 2.1** Current System Limitations

| **Platform** | **Limitations** |
|---|---|
| **CloudPOS** | Designed for multi-branch retail operations; feature overload including batch expiry tracking, chair utilization logic, and FBR integration; expensive for small single-location salons; requires technical training to use effectively; steep learning curve for users with basic IT literacy. |

| | |
|---|---|
| **MyPOS** | Primarily a POS/billing system with no customer-facing online booking portal; customers cannot book appointments online without calling; lacks automated appointment reminders; no customer dashboard for viewing booking history; requires POS terminal hardware. |
| **Websol POS** | Point-of-sale focused with no online appointment booking system; customers must schedule via phone or in-person; no customer dashboard for booking history; no automated reminders or confirmations; basic reporting only; lacks staff scheduling tools. |
| **Manual System** | No digital backup; data loss risk if records lost or damaged; no real-time synchronization causing double-bookings; no online booking; no automated reminders; no staff commission tracking; no revenue analytics. |

## 2.4 Summary

The Salon Management System project addresses the challenges faced by salon owners and customers in managing appointments and salon operations by offering a centralized, structured, and user-friendly platform. Existing local tools like CloudPOS, InstaCare, MyPOS, Websol POS, and SalonFlow either focus on complex multi-branch features, lack customer self-service portals, require expensive POS hardware, or do not offer online booking for customers. International platforms like Booksy, Vagaro, Fresha, and Square Appointments rely on high USD subscription fees, lack local payment integration, or are not available in Pakistan. Traditional manual methods using paper registers, scattered WhatsApp messages, and Excel sheets make it difficult for salon owners to manage bookings, track customer history, coordinate staff schedules, or process online payments.

The Salon Management System overcomes these limitations by enabling customers to book appointments online 24/7, view real-time staff availability, receive automated email reminders, make secure payments via Stripe, and view their complete service history. Unlike CloudPOS which is designed for multi-branch operations with complex inventory features, or MyPOS and Websol POS which are POS-focused without customer self-booking, our system prioritizes simplicity and customer convenience. For salon owners, the system provides a dedicated dashboard to manage services, staff schedules with commission tracking, track revenue, view business analytics, manage expenses, and handle subscriptions without requiring technical expertise or expensive hardware.

Built using modern technologies such as Node.js, Express.js, React.js, PostgreSQL, Prisma ORM, Redis, Bull Queue, and Stripe, following Agile methodology, the Salon Management System ensures secure authentication with JWT and HTTP-only cookies, smooth navigation, responsive design for all devices, and scalable performance. The system is specifically designed for small, single-location salons in the Pakistani market, offering transparent PKR pricing, local customer support, and essential features only — making it an affordable and practical alternative to both complex local systems and expensive international platforms.

# Chapter 3
# Requirement Analysis

# 3 Requirement Analysis

## 3.1 Use Case Diagrams

In this Use case diagram, we show the behavior of our system. This enables us to visualize the different types of roles in the system and how those roles interact with the system [12].

### 3.1.1 Customer and Salon Owner Interaction

Figure 3.1 illustrates the complete use case diagram of the LuxeBOOK platform, showing interactions between three primary actors: Customer, Provider (Salon Owner), and Admin [13].

**Customer Use Cases:** The Customer performs core tasks including Register/Login, Browse Provider, Search by Location, View Provider Profile, Select Service, Choose Staff Member, Pick Appointment Slot, Confirm Booking, Make Payment, View Booking History, Favorite Provider, and Reset Password. The System provides automated services by checking staff availability and preventing double-booking during the appointment slot selection process.

**Provider Use Cases:** The Provider performs management tasks including Setup Salon Profile, Add/Edit Services, Manage Staff, Set Business Hours, View Appointments, Manage Calendar, Track Expenses, and Subscribe to Plan. These use cases enable salon owners to completely manage their business operations from a single dashboard.

**Admin Use Cases:** The Admin performs oversight tasks including viewing all providers, managing providers, and viewing system analytics. The admin ensures platform quality by monitoring provider registrations and overall system performance.

**Shared Use Cases:** Both Customer and Provider can access Register/Login and Reset Password functionalities, while the Admin also has separate login access for platform oversight.


**Figure 3.1 Use Case diagram**

## 3.2 Use Case Description:

### 3.2.1 Customer Appointment Booking

Table 3.1 defines how customers create, customize, and manage their appointments within the Salon Management System platform. It outlines the steps involved in service selection, staff selection, time slot booking, payment processing, and final confirmation of the appointment. The description ensures clarity in user interactions, system responsibilities, and automated payment processing, forming the foundation for seamless appointment generation.

**Table 3.1** Customer Appointment Booking

| | |
|---|---|
| **Use Case ID:** | UC-01 |
| **Use Case Name:** | **Customer Appointment Booking** |
| **Actors:** | Primary: Customer<br>Secondary: System (Availability Checker, Payment Processor, Email Service) |

| | |
|---|---|
| **Description:** | Customers register an account, log in, browse services, select preferred staff member, choose available time slot, make payment via Stripe, and receive booking confirmation. The system checks availability in real-time and prevents double-booking. |
| **Trigger:** | Customer clicks "Sign Up" to create an account, then clicks "Book Now" button on service page, OR an existing customer logs in and selects a service from the catalog. |
| **Preconditions:** | PRE-1. Customer must have a valid email address for registration and notifications.<br>PRE-2. Customer must be logged into the system.<br>PRE-3. Selected staff member must be active and working on the selected day.<br>PRE-4. Database and Stripe payment gateway must be available. |
| **Postconditions:** | POST-1. Customer account is created in database (if new user).<br>POST-2. Booking record is saved in database with status "CONFIRMED".<br>POST-3. Email confirmation is sent to customer.<br>POST-4. Booking appears in customer dashboard.<br>POST-5. Booking appears in provider calendar.<br>POST-6. Payment record is created in database. |
| **Normal Flow:** | 1. Customer opens the website and clicks "Sign Up".<br>2. Customer enters name, email, and password to create an account.<br>3. System sends verification email to customer (if email verification is enabled).<br>4. Customer verifies email and logs into the system.<br>5. Customer navigates to the services page.<br>6. System displays list of available services with prices and durations.<br>7. Customer selects a service from the list.<br>8. System displays staff members qualified for the selected service.<br>9. Customer selects a preferred staff member.<br>10. System checks staff schedule and displays available time slots.<br>11. Customer selects an available time slot.<br>12. System displays booking summary with service, staff, time, and total |

| | |
|---|---|
| | amount.<br>13. Customer clicks "Proceed to Payment".<br>14. System creates a Stripe payment intent and displays payment form.<br>15. Customer enters card details and confirms payment.<br>16. System processes payment via Stripe API.<br>17. Upon successful payment, system creates booking record in database.<br>18. System adds email job to Bull queue for confirmation email.<br>19. System displays booking confirmation page with booking ID.<br>20. Customer receives email confirmation with appointment details. |
| **Alternative Flows:** | 1.1 Existing customer logs in directly instead of registering.<br>1. Customer clicks "Sign In" instead of "Sign Up".<br>2. Customer enters email and password.<br>3. Return to step 5 of normal flow.<br>1.2 Customer edits an existing booking instead of creating a new one.<br>1. Return to step 10 of normal flow to select new time slot.<br>1.3 Selected time slot becomes unavailable during booking.<br>1. Systems displays "Time slot no longer available" message.<br>2. System refreshes available time slots.<br>3. Return to step 10 of normal flow.<br>1.4 Customer has favorite services saved.<br>1. System displays favorite services on dashboard.<br>2. Customer selects from favorites.<br>3. Proceed to step 8 of normal flow. |
| **Exceptions:** | E1. If a required field is missing during registration, the system prompts the customer.<br>1. If a required field is missing, then the system prompts the customer to complete it.<br>2a. If the customer provides the missing information, then the system proceeds with normal flow.<br>2b. Else if the customer cancels registration, then the system terminates the use case.<br>E2. If email already exists during registration. |

| | |
|---|---|
| | 1. System displays "Email already registered" message.<br>2. Customer can click "Sign In" instead.<br>E3. If Stripe payment fails or is declined.<br>1. System displays "Payment failed - please try again" message.<br>2. Booking remains in pending state.<br>3. Return to step 13 of normal flow for retry.<br>E4. If database connection fails during booking.<br>1. System displays "Service temporarily unavailable" error message.<br>2. No booking is created.<br>3. User is asked to try again later. |
| **Business Rules** | BR-1. Customer must have a verified account to book appointments.<br>BR-2. A customer cannot book more than one appointment at the same time.<br>BR-3. A staff member cannot have overlapping appointments.<br>BR-4. Payment must be completed before booking is confirmed.<br>BR-5. Appointments cannot be booked for past dates.<br>BR-6. Minimum booking notice is 1 hour before appointment time.<br>BR-7. Customers can cancel bookings up to 2 hours before appointment without penalty.<br>BR-8. Each booking must be associated with exactly one service and one staff member.<br>BR-9. Booking status changes from PENDING to CONFIRMED only after successful payment. |
| **Assumptions:** | 1. Customer has a valid email address for registration and notifications.<br>2. Customer has a valid payment method (credit/debit card) for Stripe payments.<br>3. Customer has basic internet browsing knowledge.<br>4. Stripe payment gateway is available during booking.<br>5. Customer will receive email within a few minutes after registration and booking. |

### 3.2.2 Provider Dashboard and Analytics

Table 3.2 explains how the Salon Management System Dashboard analyzes business data to generate meaningful revenue insights, identify booking patterns, and show relevant performance metrics. It highlights the interaction between the Salon Owner and the System as it processes appointment, payment, and staff-based information. The description also outlines how personalized analytics are calculated based on real-time booking data and customer behavior, ensuring continuous and actionable business guidance.

**Table 3.2** Provider Dashboard

| **Use Case ID:** | UC-02 |
|---|---|
| **Use Case Name:** | **Provider Dashboard and Analytics** |
| **Actors:** | Primary: Provider (Salon Owner)<br>Secondary: System (Analytics Engine, Database) |
| **Description:** | System analyzes appointment and payment data, then displays revenue analytics, booking statistics, popular services, and staff performance metrics to help salon owners make informed business decisions. |
| **Trigger:** | Provider logs into dashboard and clicks "Analytics" tab, OR dashboard loads automatically after login. |
| **Preconditions:** | PRE-1. Provider must be logged into the system.<br>PRE-2. Provider account must be approved by admin.<br>PRE-3. Provider must have at least one completed booking in the database to show meaningful analytics. |
| **Postconditions:** | POST-1. System displays revenue analytics dashboard with charts.<br>POST-2. Booking statistics are calculated and shown.<br>POST-3. Popular services are ranked and displayed.<br>POST-4. Staff performance metrics are calculated and shown. |
| **Normal Flow:** | 1. 1. Provider logs into the system using email and password.<br>2. System redirects to provider dashboard.<br>3. System queries database for all bookings belonging to this provider.<br>4. System calculates total revenue for today, this week, and this month. |

| | |
|---|---|
| | 5. System calculates total number of bookings (total, completed, cancelled, no-show).<br>6. System identifies popular services by counting bookings per service.<br>7. System calculates staff performance metrics:<br>- Total appointments completed per staff member<br>- Total revenue generated per staff member<br>- Commission earned per staff member<br>8. System generates charts and graphs for visual representation.<br>9. System displays analytics dashboard to provider.<br>10. Provider views revenue trends, booking patterns, and staff performance.<br>11. Provider can select date range (today, week, month, custom) to filter data.<br>12. System updates charts based on selected date range. |
| **Alternative Flows:** | 1.1 Provider exports analytics report.<br>1. Provider clicks "Export Report" button.<br>2. System generates PDF or CSV file with all analytics data.<br>3. System downloads file to provider's device.<br>4. Return to step 10 of normal flow.<br>1.2 Provider has no bookings yet.<br>1. System displays "No data available yet. Complete your first booking to see analytics." message.<br>2. System shows empty charts with zero values.<br>1.3 Provider views expense tracking instead of revenue analytics.<br>1. Provider clicks "Expenses" tab in dashboard.<br>2. System displays expense list and summary.<br>3. Provider can add, edit, or delete expenses. |
| **Exceptions:** | E1. If database query fails during analytics calculation.<br>1. System displays "Unable to load analytics. Please try again later." error message.<br>2. System logs the error for debugging.<br>3. Provider can refresh the page to retry. |

| | |
|---|---|
| | E2. If no bookings exist for selected date range.<br>1. System displays "No bookings found for selected date range" message.<br>2. Charts show zero values.<br>3. Provider can select a different date range. |
| **Business Rules** | BR-1. Revenue calculations include only completed and paid bookings.<br>BR-2. Cancelled and no-show bookings are excluded from revenue totals.<br>BR-3. Popular services are ranked by number of bookings (highest first).<br>BR-4. Staff commissions are calculated as (booking amount × commission percentage / 100).<br>BR-5. Analytics data is updated in real-time as new bookings are made.<br>BR-6. Providers can only view data for their own salon (not other providers).<br>BR-7. Expense tracking is separate from revenue calculations. |
| **Assumptions:** | 1. Provider has completed at least one booking to see meaningful analytics.<br>2. Provider has set commission rates for staff members.<br>3. Database has accurate payment records for revenue calculation.<br>4. Provider understands basic chart interpretation.<br>5. System timezone is set correctly for date-based calculations. |

### 3.2.3 Role Based Access Control and Privacy

Table 3.4 describes how the Salon Management System manages user access, authentication, and data privacy based on user roles within the platform. It explains how the system implements role-based access control (RBAC), ensures secure authentication via JWT tokens, and supports HTTP-only cookies for session management. The description also highlights built-in safeguards to prevent unauthorized access and maintain data protection standards for customers, providers, and administrators.

**Table 3.3** Sharing and Privacy Controls

| | |
|---|---|
| **Use Case ID:** | UC-03 |
| **Use Case Name:** | Role-Based Access Control and Privacy |
| **Actors:** | Primary: Customer, Provider, Admin<br>Secondary: System (Authentication Middleware, Authorization Middleware) |
| **Description:** | System manages user access based on assigned roles (Customer, Provider, Admin). Each role can only access features and data specifically permitted for that role. JWT tokens stored in HTTP-only cookies ensure secure authentication for all requests. |
| **Trigger:** | User attempts to access a protected route or API endpoint, OR user logs into the system. |
| **Preconditions:** | PRE-1. User must be registered in the system with a valid role.<br>PRE-2. User must be logged in with valid credentials.<br>PRE-3. JWT token must be valid and not expired. |
| **Postconditions:** | POST-1. User is granted or denied access based on role permissions.<br>POST-2. Access attempts are logged for security auditing.<br>POST-3. Unauthorized access attempts return 403 Forbidden error. |
| **Normal Flow:** | 1. User submits login credentials (email and password).<br>2. System validates credentials against database.<br>3. If valid, system generates JWT token with user ID and role embedded.<br>4. System stores JWT token in HTTP-only cookie (not accessible via JavaScript).<br>5. User attempts to access a protected route or API endpoint.<br>6. Browser automatically sends cookie with the request.<br>7. authMiddleware.js intercepts the request and extracts JWT token from cookie.<br>8. authMiddleware validates token signature and checks expiration.<br>9. authMiddleware attaches user object (id and role) to req.user.<br>10. authorize([roles]) middleware checks if req.user.role is in allowed roles list.<br>11. If role matches, request proceeds to controller. |

| | |
|---|---|
| | 12. If role does not match, system returns 403 Forbidden error.<br>13. User accesses only features permitted for their role. |
| **Role-Specific Access Rules:** | **Customer Access:**<br>Can view and book services<br>Can view own appointments only<br>Can make payments for own bookings<br>Can view own profile and booking history<br>Cannot view other customers' data<br>Cannot access provider or admin features<br><br>**Provider Access:**<br>Can manage own salon services<br>Can manage own staff members<br>Can view appointments for own salon only<br>Can view own revenue analytics<br>Cannot view other providers' data<br>Cannot access customer or admin features<br><br>**Admin Access:**<br>Can view all providers and customers<br>Can approve or reject provider registrations<br>Can view system-wide analytics<br>Can manage user accounts (disable/delete)<br>Has full system access for oversight |
| **Alternative Flows:** | 1.1 User attempts to access a page without logging in.<br>1. System redirects user to login page.<br>2. User must log in to continue.<br>1.2 JWT token has expired.<br>1. System returns 401 Unauthorized error.<br>2. User is redirected to login page.<br>3. User must log in again to receive new token.<br>1.3 User logs out of the system.<br>1. System clears the HTTP-only cookie containing JWT token. |

| | |
|---|---|
| | 2. User is redirected to login page.<br>3. User must log in again to access protected routes. |
| **Exceptions:** | E1. If JWT token is tampered with or invalid signature.<br>1. authMiddleware detects invalid signature.<br>2. System returns 401 Unauthorized error.<br>3. User is redirected to login page.<br>E2. If database connection fails during authentication.<br>1. System returns "Service temporarily unavailable" error.<br>2. User cannot log in until database is available.<br>E3. If user role is not set in the database.<br>1. System returns "Invalid user role" error.<br>2. User cannot access any features until role is assigned. |
| **Business Rules** | BR-1. JWT tokens must be stored in HTTP-only cookies (not local Storage).<br>BR-2. Tokens expire after 7 days and users must log in again.<br>BR-3. Passwords must be hashed using bcrypt before database storage.<br>BR-4. All protected API endpoints must use authMiddleware first.<br>BR-5. authorize([roles]) middleware must be used after authMiddleware for role-specific endpoints.<br>BR-6. Customers cannot access provider or admin routes.<br>BR-7. Providers cannot access admin routes or other providers' data.<br>BR-8. Admin users have access to all system features.<br>BR-9. All authentication attempts (success and failure) must be logged.<br>BR-10. User sessions are stateless (no session storage on server). |
| **Assumptions:** | 1. Users have valid email addresses for registration.<br>2. Users understand their role and associated permissions.<br>3. Browser supports HTTP-only cookies.<br>4. Users will log out after completing their session.<br>5. Admin users are trustworthy and have been properly authorized. |

## 3.3 Functional Requirements

Functional requirements are essential in software development because they clearly specify what the system must do to fulfill users' needs and expectations. These requirements define the exact features, operations, and behaviors the system should provide, such as user authentication, appointment booking, payment processing, staff management, and business analytics. Their importance lies in establishing a shared understanding among developers, users, and stakeholders, ensuring that the system's capabilities are fully aligned with project goals and user expectations. By identifying these functions early in the development lifecycle, functional requirements help avoid misunderstandings, minimize costly revisions, improve planning accuracy, and support thorough testing and validation. Moreover, they act as a foundation for system design, implementation, and evaluation. Ultimately, functional requirements guide the overall development process and ensure that the final product delivers meaningful, reliable, and user-centered value.

The functional requirements of the Salon Management System encompass a wide range of features and capabilities focused on enabling users to efficiently book appointments, manage salon operations, process payments, and access business analytics. These requirements are designed to support customers, salon owners, and administrators through structured workflows and automated tools. Each function contributes to improving efficiency, visibility, and engagement within the salon business ecosystem. The key functional requirements include the following:

### 3.3.1 User Authentication and Authorization

The system shall provide a secure login and registration mechanism for customers, salon owners (providers), and administrators. It will implement role-based access control to ensure that each user type can only access features relevant to their role within the Salon Management System platform. This separation of permissions enhances security, prevents unauthorized access, and ensures a personalized user experience. Additionally, the authentication system will support password recovery, email verification, and session management to improve usability and reliability.

### 3.3.2 Appointment Booking Management

The platform allows customers to browse available services, view staff member qualifications, check real-time availability, and book appointments online without calling the salon. Users can select services, choose preferred staff members, pick available time slots, and make secure payments through Stripe. The system will prevent double-booking by checking availability in real-time before confirming appointments. Customers can also view their upcoming appointments, cancel or reschedule bookings, and receive email confirmations and reminders.

### 3.3.3 Payment Processing

The system shall enable secure online payments through Stripe payment gateway integration. When a customer initiates checkout, the system creates a payment intent and displays a secure payment form. Upon successful payment, the system automatically updates booking status from pending to confirmed and generates an invoice. Stripe webhooks will handle asynchronous payment confirmations to ensure reliable processing. Failed payments will keep the booking pending and notify the customer to retry.

### 3.3.4 Staff Management

The platform shall allow salon owners to add, edit, and remove staff members working at their salon. For each staff member, owners can set name, role, contact information, commission rate, and weekly working hours including break times. Staff schedules will be visible to customers during the booking process. Commission rates will be automatically calculated based on completed services. Owners can approve or reject staff time-off requests and track staff performance metrics including completed appointments and revenue generated.

### 3.3.5 Service Management

The system shall allow salon owners to create, update, and delete services offered by their salon. Each service will include name, description, price, duration in minutes, and category (e.g., haircut, spa, massage, makeup). Services can be assigned to specific staff members who are qualified to perform them. Service prices can be updated at any time without affecting existing bookings. Services can be temporarily disabled during holidays or unavailability. All active services will be displayed to customers for online booking.

### 3.3.6 Provider Dashboard and Analytics

The platform shall provide a comprehensive dashboard for salon owners showing revenue analytics and business performance. Owners will see total revenue for today, this week, and this month on the dashboard. Booking statistics will show total bookings, completed appointments, cancellations, and no-shows. Popular services will be displayed in charts showing which services are booked most frequently. Staff performance metrics will show each staff member's completed appointments and revenue generated. Owners can also track business expenses by category (rent, utilities, salaries, products) and generate exportable reports in PDF or CSV format.

### 3.3.7 Customer Dashboard

The system shall provide a dashboard for customers showing all their appointment and payment information. Upcoming appointments will be displayed with date, time, service name, staff member, and salon address. Booking history will show all past appointments with status (completed, cancelled, or no-show). Customers can cancel or reschedule upcoming appointments directly from their dashboard. Payment history will show all transactions with amount, date, and payment status. Favorite services can be saved for quick booking in the future. Profile settings allow customers to update their contact information and password.

### 3.3.8 Admin Panel

The system shall provide an admin panel for platform supervision and management. Admins can view a complete list of all registered salon owners and customers. New provider registrations will appear with pending status requiring admin approval. Admins can approve or reject provider registrations based on business verification. All subscription plans and active subscriptions can be viewed and managed from the admin panel. System-wide analytics will show total users, total bookings, total revenue, and platform growth metrics. Admins can disable or delete user accounts in case of policy violations.

### 3.3.9 Automated Notifications

The platform shall implement automated email notifications using Bull queue and Redis for reliable background processing. Customers will receive a booking confirmation email immediately after successful payment. An appointment reminder email will be sent 24 hours before the scheduled appointment time. Payment receipts will be emailed to customers after each successful transaction. Salon owners will receive email notifications when a new booking is made for their salon. Bull queue ensures retry mechanisms if email delivery fails temporarily. All notification templates will be customizable through the admin panel.

### 3.3.10 Subscription Management

The system shall implement subscription plans for salon owners with three tiers: Basic, Premium, and Enterprise. The Basic plan will include essential features such as booking and service management. The Premium plan will add advanced analytics, expense tracking, and priority support. The Enterprise plan will include all features plus multi-location support and dedicated account manager. Subscription payments will be processed through Stripe on a monthly or yearly basis. Subscription status will be checked before allowing access to premium features. Expired subscriptions will automatically demote owners to a limited free tier. Owners will receive email reminders before their subscription expiration date.

#### 3.3.11 Expense Tracking Module

The platform shall allow salon owners to track business expenses for complete financial reporting. Owners can add expenses with fields for amount, date, category (rent, utilities, product purchases, staff salaries, marketing, maintenance), and description. Each expense entry can be edited or deleted after creation. Monthly and yearly expense summaries will be displayed in the analytics dashboard. Expense reports can be exported as PDF or CSV for accounting purposes. Charts will show expense breakdown by category for easy visualization. Total expenses will be automatically deducted from revenue to show net profit.

#### 3.3.12 Review and Rating System

The system shall enable customers to leave reviews and ratings for services they have received. After an appointment is marked as completed, customers will receive an email invitation to leave a review. Ratings will be on a 5-star scale, with 1 being poor and 5 being excellent. Customers can write a text comment describing their experience with the service and staff. All reviews will be visible on service pages and staff profile pages. Salon owners can respond to customer reviews publicly on the platform. Average ratings will be calculated and displayed next to each service and staff member. Reviews cannot be edited or deleted by customers after submission to maintain authenticity.

### 3.4 Nonfunctional Requirements

Nonfunctional requirements define the quality attributes, constraints, and performance standards that determine how the Salon Management System operates. These requirements focus on system behavior rather than specific functionalities, ensuring reliability, efficiency, and positive user experience. They play a critical role in maintaining scalability, security, and usability as the platform grows. The key nonfunctional requirements are outlined below:

#### 3.4.1 Performance

The platform shall ensure fast loading times across all core features, including service pages, appointment booking, payment processing, and provider dashboards. Database queries for availability checking and booking creation must execute efficiently without noticeable delays. The system should be optimized to handle multiple concurrent users, especially during peak business hours when several customers may book appointments simultaneously. Consistent performance is essential to maintaining user trust and engagement. Page load time shall be less than 3 seconds under normal network conditions. Booking requests (excluding payment) shall be processed within 500 milliseconds.

### 3.4.2 Security

The system shall safeguard user data through strong encryption, secure authentication mechanisms, and strict access control policies. Customer personal information, payment details, and booking records must be securely stored and transmitted. Passwords shall be hashed using bcrypt before being stored in the database. JWT tokens shall be stored in HTTP-only cookies to prevent XSS attacks. The platform will implement role-based access control (RBAC) to ensure customers can only access their own data, providers can only access their salon data, and admins have appropriate oversight access. All API endpoints shall validate user roles before processing requests. Regular security checks and dependency updates will further reduce the risk of unauthorized access or data breaches.

### 3.4.3 Availability

The platform should maintain high availability to ensure users can access services during business hours. Customers should be able to book appointments, make payments, and view their booking history without interruption. Providers should be able to manage services, staff schedules, and view analytics reliably. The system shall have automatic database backups performed daily to prevent data loss. In case of Stripe API downtime, the system shall display user-friendly error messages and allow users to retry later. Bull queue retry mechanisms shall handle failed email deliveries without losing notification requests. This requirement ensures continuity of salon booking operations.

### 3.4.4 Ease of Use

The platform's interface should be intuitive and user-friendly for customers, salon owners, and administrators with varying levels of technical expertise. Navigation must be simple, consistent, and well-structured, with clear labels and visually clean layouts using Bootstrap 5. The system will support responsive design for desktop, tablet, and mobile devices so users can book appointments from any device. Error messages shall be clear and helpful, suggesting solutions when possible (e.g., "Time slot not available please select another time"). Confirmation dialogs shall be shown before destructive actions like cancelling appointments. Future enhancements may include Urdu language support to further broaden usability for local Pakistani users.

# Chapter 4
# Design and Architecture

# 4 Design and Architecture

## 4.1 System Architecture

Figure 4.1 explains how LuxeBOOK operates by describing its core components, how they are organized, and the way they interact with each other to ensure smooth functionality throughout the platform.

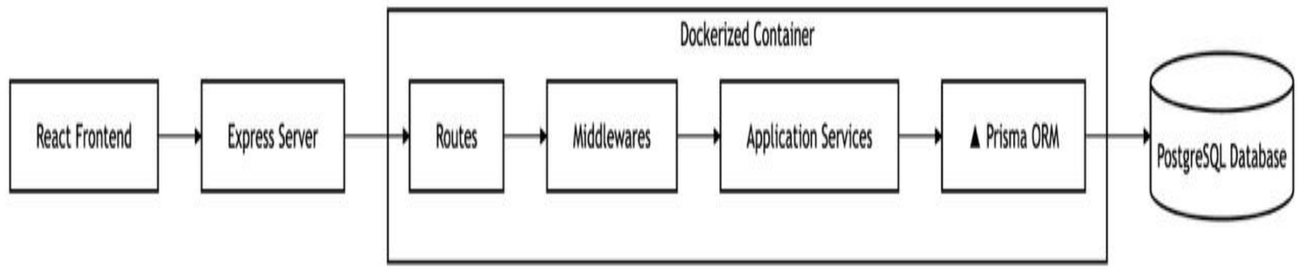



**Figure 4.1 System Architecture**

The architecture of the LuxeBOOK platform follows a modern client–server model designed to be modular, scalable, and easy to maintain. The client side is developed using React.js, HTML5, CSS3, JavaScript, Bootstrap 5, and Axios, providing fast, responsive, and intuitive interfaces for customer booking, provider dashboard, service browsing, appointment management, and payment processing. The frontend communicates with the backend through RESTful API calls made via Axios, ensuring smooth data exchange between the user interface and server.

The backend is powered by Node.js 20.x and Express.js 4.x, where the business logic manages core functionalities such as user authentication, appointment booking, payment processing, staff management, service management, subscription handling, and automated notifications, ensuring a clean separation between the interface and application logic. The backend follows a layered architecture with routes, controllers, middleware, and services. Key middleware includes authentication middleware (JWT verification), authorization middleware (role-based access control), and error handling middleware.

The system uses PostgreSQL as the primary relational database with Prisma ORM 5.x for type-safe database operations, schema management, and migrations. Prisma Client provides an intuitive API for database queries, reducing SQL boilerplate code and ensuring data consistency. The database stores user profiles, customer and provider information, service catalogs, staff details, appointment bookings, payment records, subscription plans, expenses, and customer reviews.

For caching and background job processing, the platform integrates Redis as an in-memory data store and Bull Queue for handling asynchronous tasks. Email notifications including booking confirmations, appointment reminders (24 hours before scheduled time), and payment receipts are processed through Bull Queue with Redis as the message broker, ensuring reliable delivery with retry mechanisms for failed emails.

The platform includes automated cron jobs using node-cron for scheduled tasks such as cancelling expired pending bookings (every 15 minutes), deleting old completed bookings, sending appointment reminders, and checking subscription expiry (daily at midnight). Role-Based Access Control (RBAC) ensures that customers, providers, and admins can only access features relevant to their roles, while secure API communication via HTTPS and encrypted storage protect sensitive user data including passwords hashed with bcrypt and JWT tokens stored in HTTP-only cookies.

LuxeBOOK's modular architecture supports future enhancements and scalability, allowing additional modules such as inventory management, loyalty programs, mobile applications, and multi-branch support to be integrated smoothly. This structure ensures efficient performance, maintainability, and a seamless user experience across all components of the platform.

## 4.2 Data Representation

In the LuxeBOOK platform, all user, appointment, service, staff, payment, and subscription-related data are stored within structured relational databases to ensure fast, consistent, and reliable access to information. The system utilizes PostgreSQL with well-defined relational schemas to systematically organize user profiles, customer and provider information, service catalogs, staff details, appointment bookings, payment records, subscription plans, expenses, and customer reviews while maintaining strong relationships among these entities. Unlike simple document-based or unstructured storage systems, this relational approach enables complex joins, indexing, and constraint enforcement, which are essential for maintaining data integrity and reducing redundancy.

The structured data model allows LuxeBOOK to support advanced querying mechanisms required for real-time availability checking, double-booking prevention, revenue analytics, and staff performance tracking. Efficient data encoding and decoding facilitate real-time updates, seamless synchronization of booking data, and responsive provider dashboard queries. Furthermore, the relational design ensures smooth integration with Prisma ORM by providing clean, normalized, and query-optimized datasets. As a result, this approach significantly enhances system accuracy, scalability, and performance, ultimately delivering a faster, more reliable, and user-centric experience across the platform [18].

## 4.3 Process Flow

Figure 4.2 Process Flow Representation for LuxeBOOK illustrates the complete journey of customers, providers (salon owners), and administrators as they navigate the platform. It outlines how customers register, browse providers, select services, choose staff members, book appointments, make payments, and receive confirmations. The flow also shows how customers manage their appointments, view booking history, and maintain a structured booking record that can be accessed from their dashboard.

Providers follow a parallel process where they register, set up their salon profile, add services, manage staff, handle bookings, track expenses, and manage subscriptions. Admins follow a simplified process where they log in, view customers and providers, and monitor the overall system. The diagram demonstrates how all three user types interact smoothly with the system, highlighting an intuitive workflow and clear separation of roles. Overall, it captures how LuxeBOOK brings efficiency, clarity, and seamless communication between customers and salons, ultimately creating a user-friendly environment that supports salon visibility and seamless appointment booking.

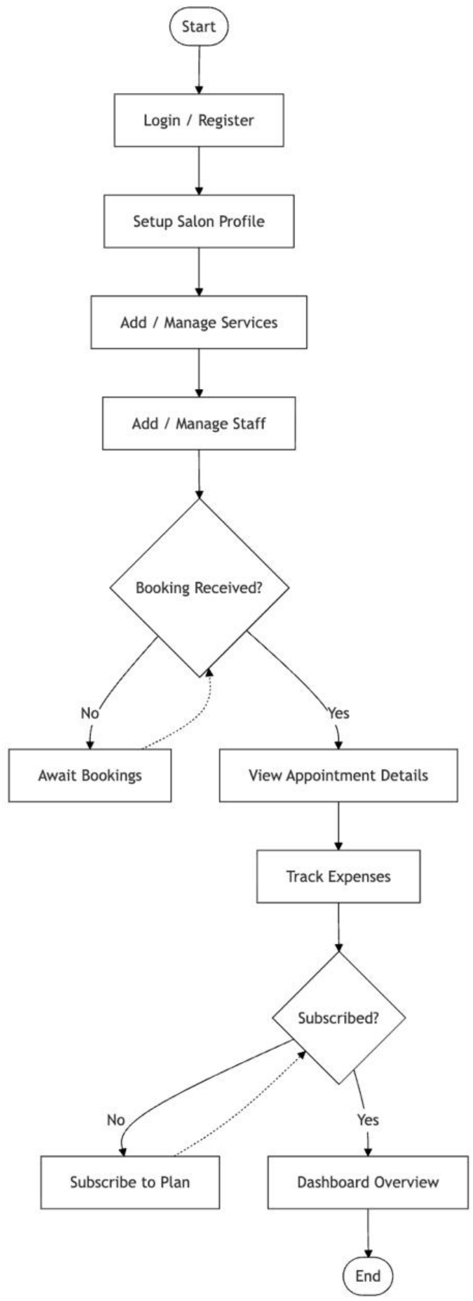



**Figure 4.2 Customer flow**

1. **User Registration and Login**

Users begin by creating an account and logging into the platform. After successful authentication using JWT tokens stored in HTTP-only cookies, they are redirected to their personalized dashboard based on their role (Customer, Provider, or Admin).

2. **For Customers**

   **Browse and Search Providers:** Customers can search for salons by name, browse service categories, and view provider profiles with ratings and reviews.

**View Provider Profile:** Customers can view salon details, service menus, staff qualifications, pricing, and customer reviews before making a decision.

**Select Service(s):** Customers select desired services from the provider's service catalog, including price and duration information.

**Choose Staff Member:** Customers can select a preferred staff member based on their qualifications, ratings, and availability.

**Select Available Slot:** The system displays real-time available time slots based on staff schedules and existing bookings. If a slot is unavailable, the customer must pick another slot.

**Confirm Booking:** Once a time slot is selected, the customer proceeds to confirm the booking details.

**Make Payment (Stripe):** The customer makes a secure payment through Stripe payment gateway. If payment fails, the customer is prompted to retry. If payment succeeds, the booking is confirmed.

**Receive Confirmation:** An email confirmation is sent to the customer via Bull queue, and the booking appears in their dashboard.

**View Booking History:** Customers can view upcoming appointments, past bookings, cancel or reschedule appointments, and view payment history from their dashboard.

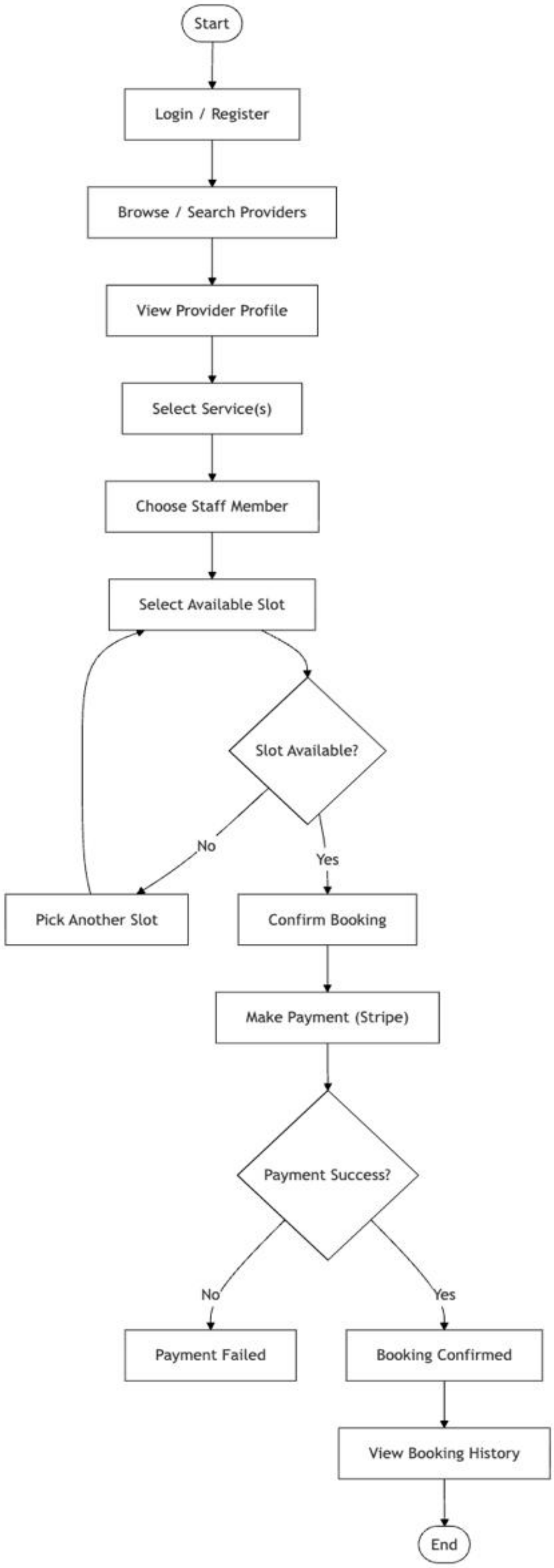



**Figure 4.3 Provider**

**3. For Providers (Salon Owners)**

**Setup Salon Profile:** Providers register and set up their salon profile including business name, address, phone number, and logo.

**Add/Manage Services:** Providers can add, edit, or delete services offered by their salon, including name, price, duration, and category.

**Add/Manage Staff:** Providers can add staff members, assign roles, set commission rates, and define working hours including break times.

**Await Bookings:** Once services and staff are set up, providers wait for customers to book appointments.

**View Appointment Details:** When a booking is received, providers can view appointment details including customer name, service, staff assigned, date, and time.

**Track Expenses:** Providers can track business expenses including rent, utilities, staff salaries, and product purchases.

**Check Subscription Status:** If the provider has an active subscription, they can access premium features including advanced analytics. If not subscribed, they are prompted to subscribe to a plan (Basic, Premium, or Enterprise).

**Dashboard Overview:** Providers can view revenue analytics, booking statistics, popular s services, staff performance metrics, and expense reports.

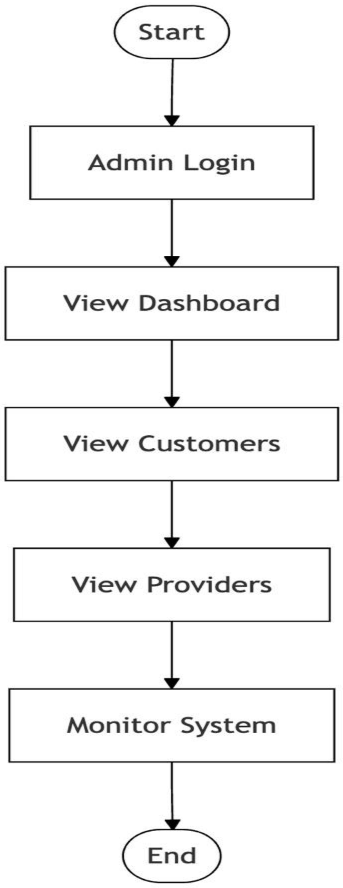



**Figure 4.4 Admin**

**4. For Administrators**

**Admin Login:** Admin logs into the system using special admin credentials.

**View Dashboard:** Admin views system-wide analytics including total users, total bookings, total revenue, and platform growth metrics.

**View Customers:** Admin can view all registered customers and their booking history.

**View Providers:** Admin can view all registered providers, approve or reject pending provider registrations, and manage provider accounts.

**Monitor System:** Admin monitors overall platform operations, manages subscriptions, and generates reports as needed

## 4.4 Sequence Diagram

### 4.4.1 Customer Registration Flow

Figure 4.3 explains the complete customer registration flow in the LuxeBOOK system. A new customer fills out the registration form with name, email, phone, and password. The backend validates the input, checks for existing email, and hashes the password using bcrypt. The system then creates a customer account in the database and returns a success response. The customer can then proceed to login. Additionally, the registration workflow is designed to handle errors gracefully, such as duplicate emails or invalid input formats, providing clear feedback to customers and maintaining a smooth onboarding experience.

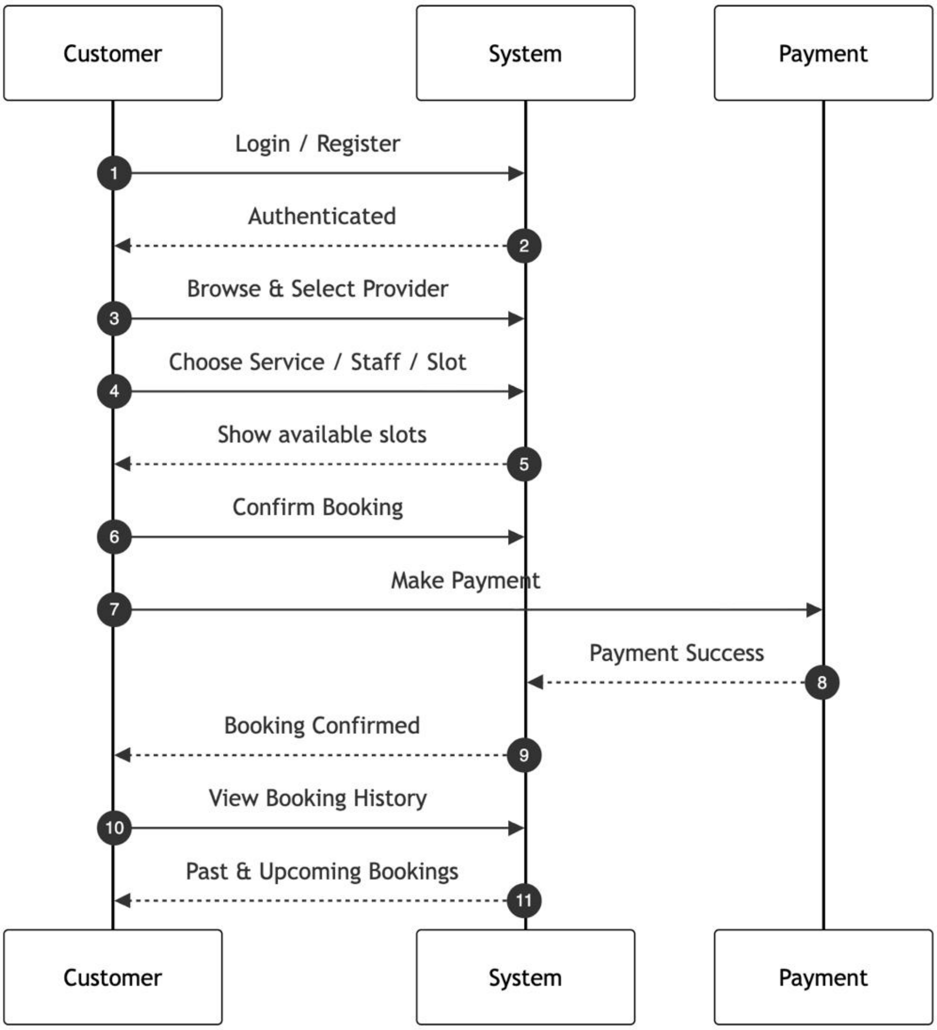



**Figure 4.5 Customer Registration**

### 4.4.2 Provider Management Flow

Figure 4.4 illustrates how a provider (salon owner) manages their salon operations in the LuxeBOOK system. The provider begins by logging in or registering into the system. After successful authentication, the provider sets up their salon profile including business name, address, phone number, and logo. The provider then adds and manages services offered by their salon, including name, price, duration, and category. Next, the provider adds and manages staff members, assigning roles, commission rates, and working hours. When a customer makes a booking, the provider receives a new booking notification. The provider can view appointment details including customer name, service selected, staff assigned, date, and time. The provider also tracks business expenses such as rent, utilities, staff salaries, and product purchases. If the provider needs access to premium features, they subscribe to a plan (Basic, Premium, or Enterprise) through the system. Finally, the provider views their dashboard overview which displays revenue analytics, booking statistics, popular services, and staff performance metrics.


**Figure 4.6 Provider Mnagement**

## 4.5 Class Diagram

Figure 4.5 Figure 4.11 illustrates the system architecture for comprehensive salon management and user management within the LuxeBOOK platform. The User class functions as the primary base entity, capturing common attributes and behaviors for all users on the platform. It is further specialized in subclasses such as Customer, Provider, and Admin, which encapsulate role-specific functionalities, permissions, and access levels. This hierarchical design ensures that each user type can perform its intended operations while maintaining a consistent underlying structure.

Each User instance maintains relationships with multiple related classes including CustomerProfile, ProviderProfile, Booking, Subscription, and Notification. The CustomerProfile class stores customer-specific information such as address, city, phone number, profile picture, and gender. The ProviderProfile class stores salon owner information including username, name, location, experience level, skills, and salary. This structured relationship allows the system to efficiently access and analyze user data for appointment booking, revenue analytics, staff management, and business insights. The design also facilitates real-time updates, ensuring that any changes in user data are immediately reflected in their dashboard and available for system processing.

Moreover, the class hierarchy promotes modularity and maintainability, allowing developers to extend or modify user roles and their associated functionalities without impacting unrelated system components. By clearly separating responsibilities across classes and establishing precise relationships between users, bookings, services, staff, and subscriptions, the architecture supports scalability, efficient data management, and seamless integration of core features. This approach not only enhances the overall performance and reliability of the platform but also provides a robust framework for future expansions, such as integrating additional user types or advanced analytics modules.

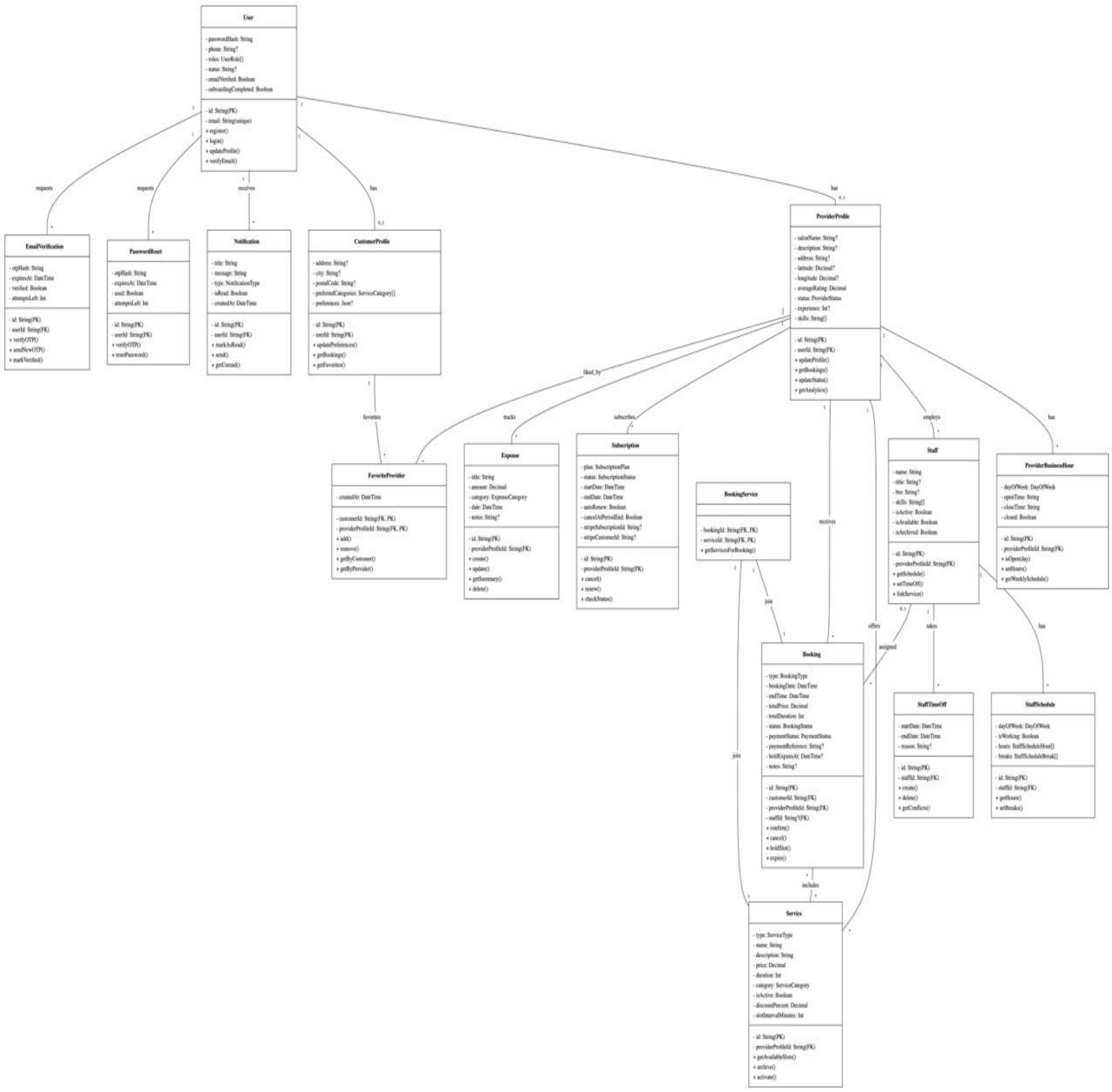


**Figure 4.7 Use Case Diagram**

### 4.5.1 Components

**User:** A User represents any individual interacting with the LuxeBOOK platform, such as a customer, provider (salon owner), or admin. Each user has a unique ID, email, and password (hashed using bcrypt), allowing secure access to the system. Users can register, log in, manage their profiles, and perform role-specific actions based on their assigned role.

**Customer:** A Customer represents a client who wants to book salon services. Each customer has profile information including address, city, postal code, phone number, profile picture, and gender. Customers can browse providers, view services, book appointments, make payments, view booking history, and leave reviews.

**Provider:** A Provider represents a salon owner or manager. Each provider has profile information including username, name, last name, location, experience level, skills, and salary. Providers can manage services, manage staff, view bookings, track expenses, manage subscriptions, and view business analytics.

**Admin:** An Admin oversees and manages the overall functioning of the platform. Each admin has a unique ID, email, and password, enabling them to perform system-level tasks. Admins can monitor and manage user accounts, approve or reject provider registrations, handle platform data, and view system-wide analytics.

**Booking:** The Booking class represents an appointment made by a customer with a provider. Each booking has a unique bookingId, service provider reference, booking date, booking time, booking price, booking status (PENDING, CONFIRMED, CANCELLED, COMPLETED, NO-SHOW), and references to the customer and service provider. Bookings are the core transactional records of the platform.

**Service:** The Service class represents a salon service offered by a provider. Each service includes type, name, description, price, duration, category, and location. Services are displayed to customers for browsing and booking. Providers can create, update, and delete their services.

**Staff:** The Staff class represents employees working at a provider's salon. Each staff member has a name, role, salary, and admin status. Staff members can be assigned to specific services, and customers can select preferred staff members during booking.

**StaffSchedule:** The StaffSchedule class defines working hours for staff members. It includes day of week, working hours, break hours, break duration, break start time, and break end time. This schedule is used for availability checking and preventing double-booking.

**StaffTimeOff:** The StaffTimeOff class tracks time-off requests for staff members. It includes start date, end date, and reason. Providers can approve or reject time-off requests.

**Subscriptions:** The Subscriptions class manages provider subscription plans. It includes plan type (Basic, Premium, Enterprise), status (ACTIVE, EXPIRED, CANCELLED), start date, end date, subscription period, and service provider details. Subscriptions control access to premium features.

**Notification:** The Notification class manages system notifications. It includes title, message, type, icon, and action ID. Notifications are sent to customers for booking confirmations and reminders, and to providers for new bookings and subscription expiry alerts.

**EventTracker:** The EventTracker class logs user activities for analytics and debugging. It includes event name, timestamp, userId, and userType. This helps track system usage and identify issues.

**EmailVerification:** The EmailVerification class handles email verification for user registration. It includes email, expiration date, verified status, and attempt failed status. This ensures only valid email addresses are used for registration.

**PasswordReset:** The PasswordReset class handles password reset functionality. It includes email, verify OTP status, and reset OTP. This allows users to securely reset forgotten passwords.

# Chapter 5
# Implementation

# 5 Implementation

In the implementation phase of the Salon Management System platform, we developed core algorithms that ensure smooth, efficient, and reliable system performance, enabling customers, salon owners, and administrators to interact seamlessly. These algorithms power essential features such as user authentication, appointment booking, payment processing, staff management, automated notifications, and secure data handling, ensuring a reliable and impactful user experience across the platform.

## 5.1 Algorithm

The core functionality of the Salon Management System platform is powered by a set of efficient algorithms implemented in Node.js 20.x using Express.js framework, Prisma ORM, and other supporting libraries. Each algorithm is optimized for accuracy, efficiency, and reliability, ensuring smooth real-time performance for customers, salon owners, and administrators while handling booking requests, payment processing, and automated notifications.

**1. User Authentication**

**Function:** authenticate_user(username, password)

**Input:** username (string), password (string)

**Output:** user (object) or error (string)

**Pseudocode:**

1. Retrieve user records from the database based on the provided username.
2. If a user with the given username exists:
   a. Compare the provided password with the stored password hash using bcrypt
   b. If the passwords match, generate JWT token and return user object
   c. If the passwords do not match, return error message "Invalid credentials"
3. If no user is found, return error message "Invalid credentials"

**2. Appointment Booking**

**Function:** bookAppointment(customerId, serviceId, staffId, date, time)

**Input** customerId (string), serviceId (string), staffId (string), date (Date), time (string)

**Output:** booking (object) or error (string)

**Pseudocode:**

1. Verify that the customer is authenticated
2. Validate booking data for required fields (service, staff, date, time)
3. Check if selected staff is working on selected date and time
4. Check for existing bookings at same staff, date, and time
5. If valid and no conflict:
    a. Create pending booking record in database
    b. Create Stripe payment intent
    c. If payment successful, update booking status to CONFIRMED
    d. Add email job to Bull queue for confirmation
    e. Return success with booking details
6. If validation fails or time slot unavailable, return error message

**3. Availability checking Algorithm**

**Function:** `checkAvailability(staffId, date, time, duration)`

**Input:** staffId (string), date (Date), time (string), duration (int)

**Output:** available (boolean) or error (string)

**Pseudocode:**

1. Retrieve staff schedule for selected date from database
2. If staff is not working on selected day, return false
3. Check if selected time falls within staff working hours
4. Query database for existing confirmed bookings at same staff, date, and time
5. Calculate booking end time based on service duration
6. Check if requested time overlaps with any existing booking
7. If no overlap and within working hours, return true with available time slots
8. If conflict exists, return false with message "Time slot not available".

**4. Staff Commission Calculation Algorithm**

**Function:** `calculateCommission(bookingAmount, commissionRate)`

**Input:** bookingAmount (float), commissionRate (float)

**Output:** commission (float)

**Pseudocode:**

1. Validate input values (bookingAmount greater than 0, commissionRate between 0 and 100)

2. Calculate commission using formula: (bookingAmount multiplied by commissionRate) divided by 100

3. Round the result to 2 decimal places for currency format

4. Return the calculated commission amount

**5. Revenue Analytics Calculation Algorithm**

**Function:** calculateRevenue(providerId, startDate, endDate

**Input:** providerId (string), startDate (Date), endDate (Date)

**Output:** totalRevenue (float)

**Pseudocode:**

1. Query database for all paid bookings belonging to given provider within date range
2. Initialize totalRevenue variable to 0
3. For each booking in the retrieved bookings list:
    a. Add booking.totalAmount to totalRevenu
4. Return totalRevenue as the final calculated amount

**5. Cron Scheduling for Subscription Expiry Algorithm**

**Function:** expireSubscriptions()

**Input:** none

**Output:** count of expired subscriptions (integer)

**Pseudocode:**

1. Get current date and time from the system

2. Query database for all active subscriptions with endDate less than current date

3. Initialize a counter variable to 0

4. For each expired subscription found:

a. Update subscription status from ACTIVE to EXPIRED

b. Find the associated provider for this subscription

c. Update provider status to INACTIVE if needed

d. Add email job to Bull queue for subscription expiry notification

e. Increment the counter by 1

5. Return the total count of subscriptions that were expired

**6. Authentication Middleware Algorithm**

**Function:** protect(req, res, next)

**Input:** req (request object with cookies), res (response object)

**Output:** next() or error response

**Pseudocode:**

1. Extract JWT token from the HTTP-only cookie in the request
2. If no token is found in the cookie:
   a. Return 401 Unauthorized response with message "Not authorized, please login"
3. If token exists:
   a. Verify the token using jwt.verify() with the JWT_SECRET key
   b. If token is valid, extract user id and role from decoded token
   c. Attach user object to req.user for use in next middleware or controller
   d. Call next() to proceed to the next middleware or route handler
4. If token verification fails (invalid or expired):
   a. Return 401 Unauthorized response with message "Invalid or expired token"

**7. Popular Services Ranking Algorithm**

**Function:** getPopularServices(providerId, limit)

**Input:** providerId (string), limit (integer, default = 5)

**Output:** popularServices (array of service objects)

**Pseudocode:**

1. Query database for all confirmed bookings belonging to given provider
2. Create a map or object to count number of bookings per service
3. For each booking in retrieved bookings:
   a. Increment the count for that booking's service
4. Sort the services by booking count in descending order (highest first)
5. Select the top N services based on the limit parameter
6. Return the ranked list of popular services with their booking counts

## 5.2 User Interface

The user interface (UI) of LuxeBOOK is designed to be simple, intuitive, and visually appealing for both customers and salon providers.

Figure 5.1 depicts the landing page of LuxeBOOK featuring a clean header with navigation links and Sign In / Create Account buttons. The main section uses the headline "BUILT FOR PREMIUM SALON BOOKINGS" with a subheading "Beautiful booking experiences for beauty and wellness brands." A search bar allows users to search by salon name, and key sections include DISCOVERY for service-based search, RATED SALONS showing top-performing providers, NEW LISTINGS for recently added providers, and TRENDING for top-rated providers in the marketplace.

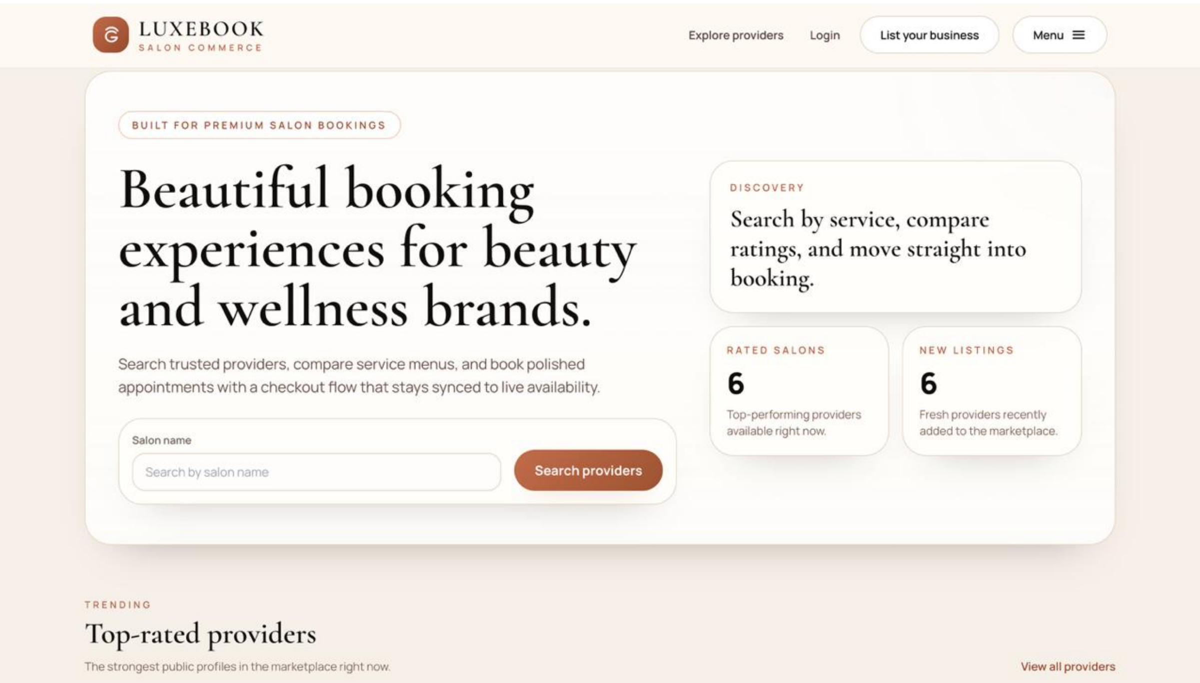



**Figure 5.1 Home Page**

Figure 5.2 depicts the login page where users can sign in to continue their session. The page offers three user role options: Customer, Provider, and Admin. Users can enter their email and password to access their workspace, with options to reset a forgotten password or create a new account. The page maintains consistent branding with the LuxeBOOK logo and "SALON COMMERCE" tagline.

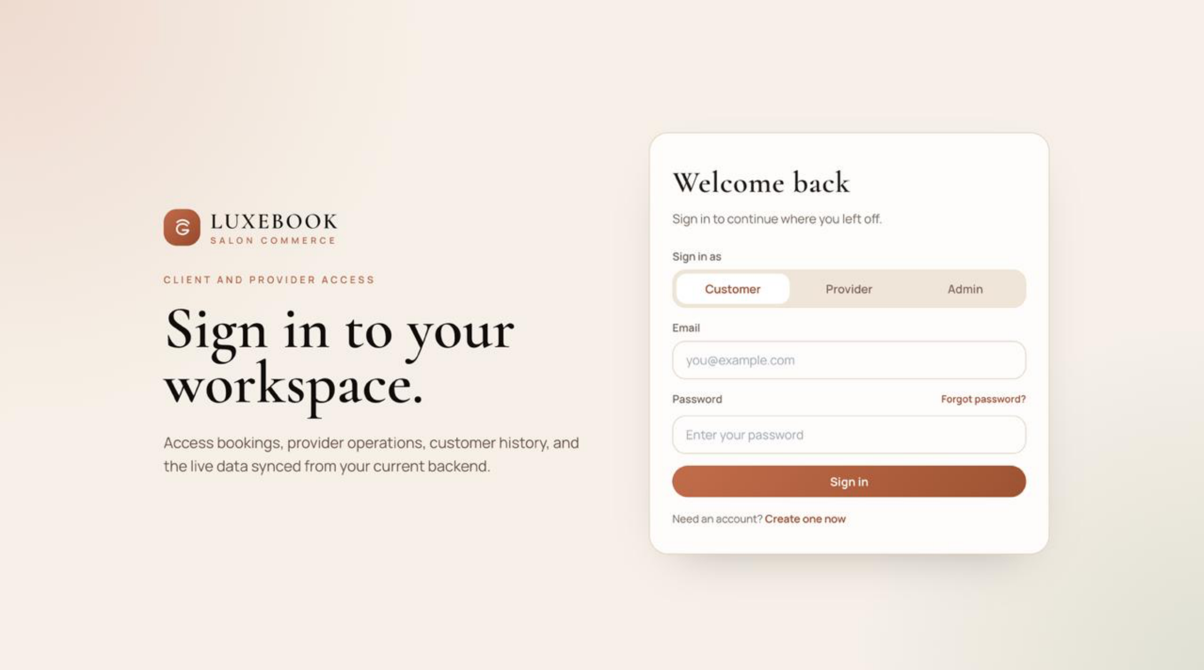



**Figure 5.2 Sign in page**

Figure 5.3 shows the provider bookings management page where salon owners can view and manage appointments. The page displays upcoming appointments including details such as date (APR 30), time (09:15 AM and 02:45 PM), customer name (Anna), service (Balayage Highlights), staff assigned (with Ayesha Khan), and price (Rs 8,500). Options to approve incoming requests, manage today's schedule, and review history are available. The sidebar navigation includes Dashboard, Bookings, Staff, Services, Subscription, Expenses, and Profile

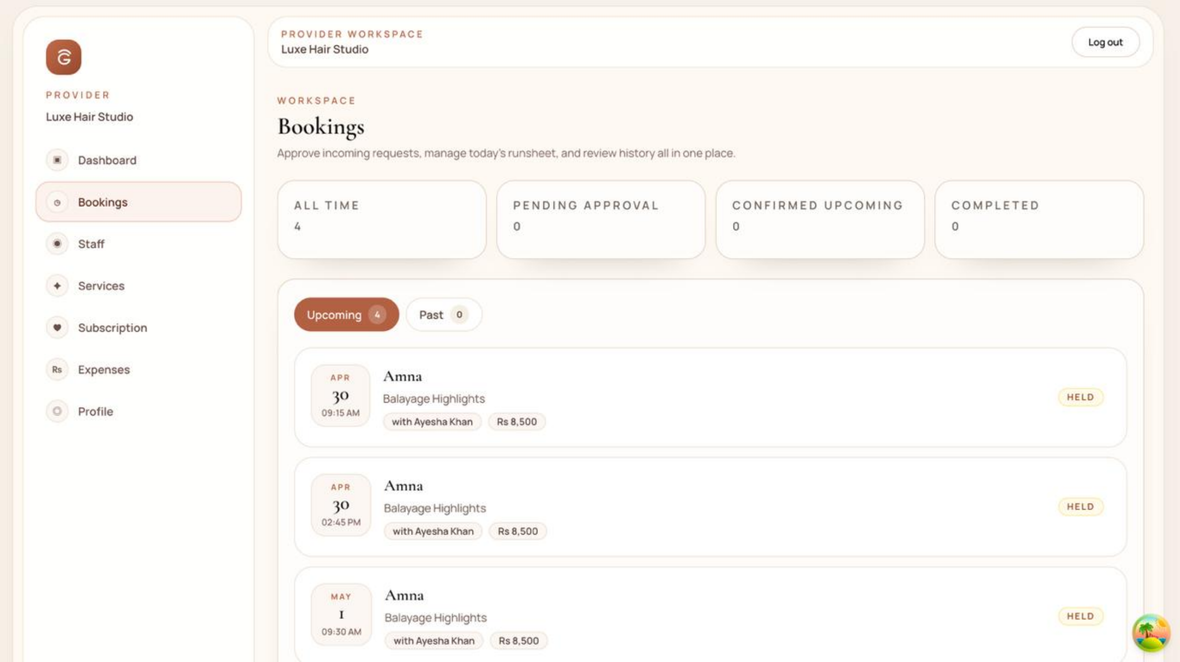


**Figure 5.3 Bookings**

Figure 5.4 illustrates the provider workspace dashboard for salon owners. The dashboard shows key metrics for the day including BOOKINGS TODAY (0), PENDING APPROVALS (0), ACTIVE STAFF (2), and PUBLISHED SERVICES (3). The access state section displays the current plan ("No Paid Plan" with status Inactive) and trial information ("15 days remaining"). Options to manage services or subscription are provided. The sidebar navigation includes Dashboard, Bookings, Staff, Services, Subscription, Expenses, and Profile options.

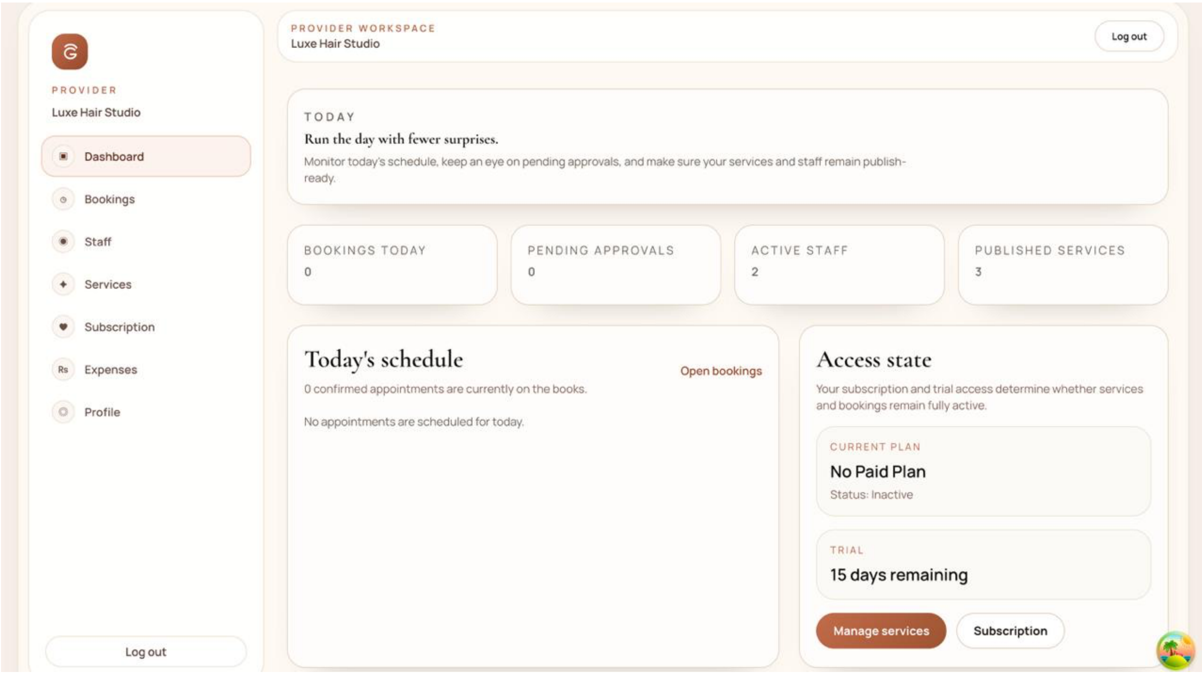



**Figure 5.4 Provider Dashboard**

Figure 5.5 depicts the services menu page where customers can browse available salon services. The page displays services including Bridal Updo (Rs 7,000, 90 MIN), Keratin Treatment (Rs 12,000, 150 MIN), and Balayage Highlights (Rs 8,500, 120 MIN). Each service card includes a description and price information. The bottom navigation includes Explore providers, Home, and List your business options.

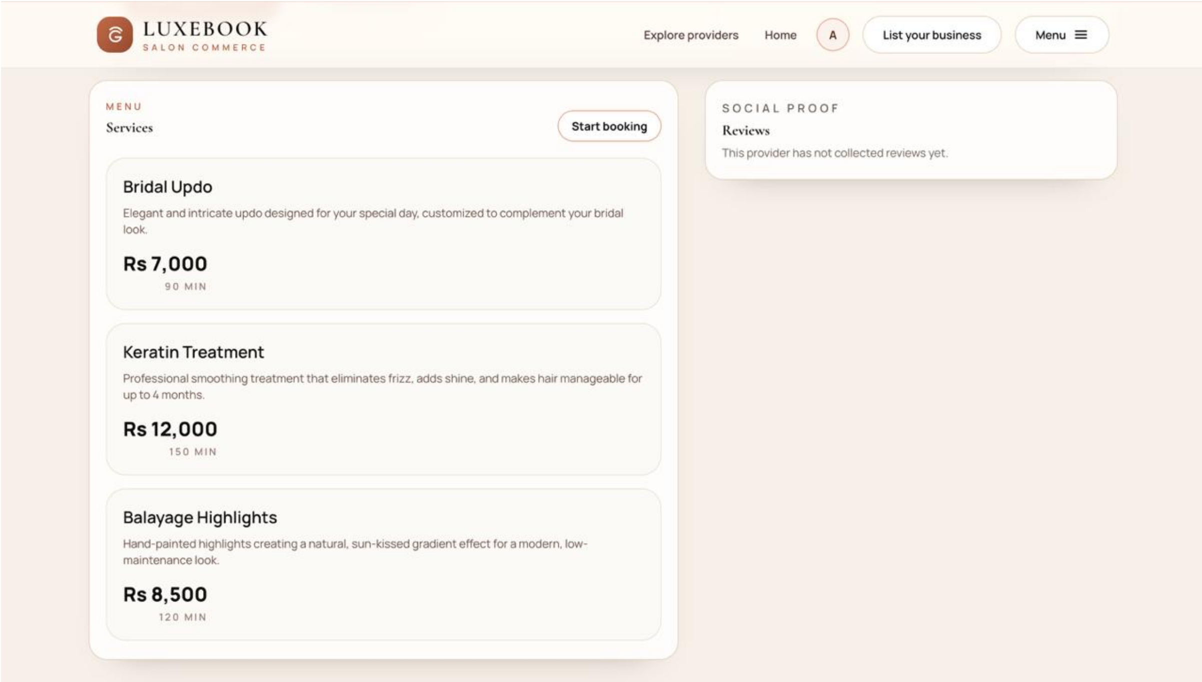



**Figure 5.5 Service menu Page**

Figure 5.6 displays the customer dashboard showing an overview of upcoming and past appointments. Key metrics include UPCOMING APPOINTMENTS (1), TOTAL APPOINTMENTS (4), and SAVED PROVIDERS (0). The dashboard allows customers to review upcoming appointments, revisit saved providers, or jump back into discovery. Navigation options include Appointments, Favorites, and Profile sections.

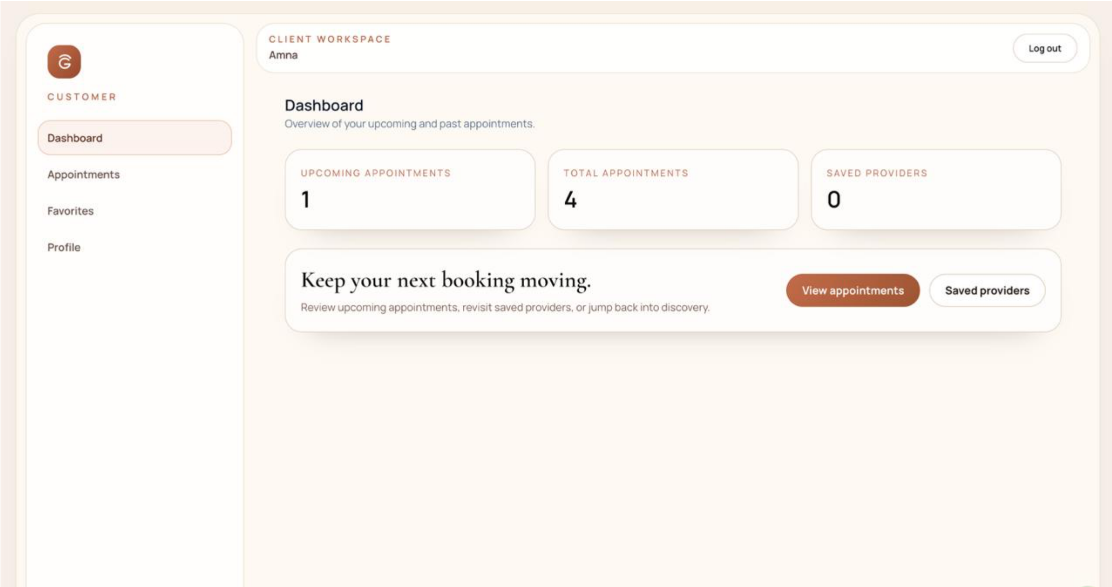
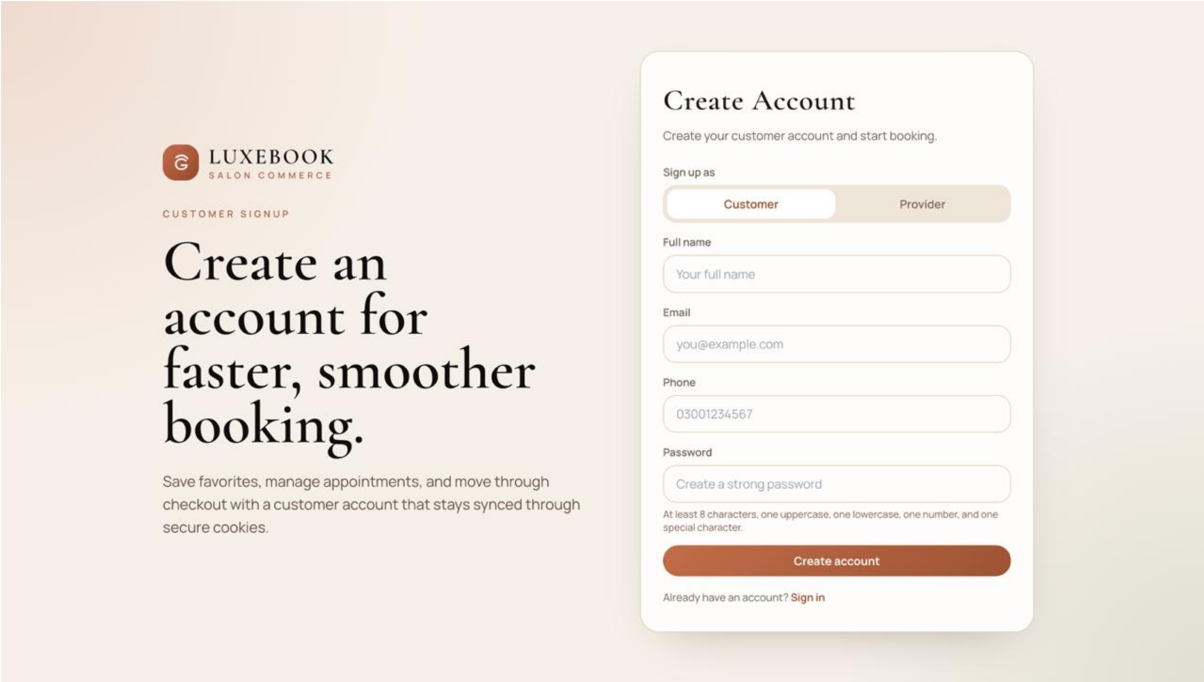



**Figure 5.6 Customer Dashboard**

Figure 5.7 presents the customer signup page designed for creating a new customer account. The page includes form fields for Full name, Email, Phone number, and Password with validation requirements (at least 8 characters, one uppercase, one lowercase, one number, and one special character). A "Create account" button allows users to complete registration, with an option to sign in if they already have an account.




## 5.3 Summary

Chapter Implementation focuses on the practical realization of the Customized Academic Chapter Implementation focuses on the practical realization of the LuxeBOOK platform, detailing how the system's architecture, algorithms, and user interface work together to deliver a seamless salon booking experience. The implementation covers core modules such as customer authentication, appointment booking, provider dashboard, staff management, service management, payment processing, and subscription handling. Efficient algorithms including JWT authentication, availability checking, double-booking prevention, commission calculation, revenue analytics, and queue-based email notifications are integrated into the backend to provide reliable and accurate operations. On the user interface side, the platform employs responsive and intuitive frontend design using React.js, HTML5, CSS3, JavaScript, Bootstrap 5, and Axios, ensuring smooth navigation across customer dashboard, provider workspace, services menu, and booking flows. Together, the backend algorithms and frontend components enable dynamic data handling, real-time availability updates, and an efficient workflow, allowing customers and salon owners to interact with the system effectively while maintaining consistency, usability, and reliable automation throughout the platform [22].

# Chapter 6
# Testing and Evaluation

# Testing and Evaluation

Testing in the Salon Management System platform plays a critical role in ensuring that all system components function correctly, efficiently, and cohesively. It verifies that core modules including user authentication, appointment booking, payment processing, staff management, and provider dashboards operate as intended and deliver smooth, reliable user experience. Testing is conducted across multiple phases of development to identify defects, inconsistencies, and usability issues at an early stage, reducing the risk of system failure after deployment.

From a software engineering perspective, testing is typically performed after implementation to allow independent evaluation of system behavior and performance. This approach ensures that the developed system aligns with defined functional and nonfunctional requirements. Through systematic testing and evaluation, the Salon Management System is validated to reliably support online booking, payment integration, staff scheduling, and business analytics, thereby meeting both customer and salon owner needs.

## 5.4 Manual Testing

Manual testing involves the execution of test cases without the use of automated tools, relying instead on direct human interaction with the system. In the Salon Management System, testers interact with the platform by assuming the roles of customers, salon owners, and administrators to simulate real-world usage scenarios. This testing approach allows evaluators to closely observe system behavior from an end-user perspective.

Manual testing is used to validate key functionalities such as appointment booking and cancellation, payment processing, staff and service management, customer dashboard navigation, and overall platform navigation. Since this method focuses on actual user interaction, it is particularly effective in identifying usability issues, interface inconsistencies, unexpected errors, and broken user flows. However, while manual testing excels at detecting surface-level and experience-related problems, certain internal logic or performance-related issues may require deeper inspection through automated or technical testing methods.

### 5.4.1 System Testing

System testing is performed to verify that the complete Salon Management System platform functions correctly as an integrated and unified system. This phase ensures that individual modules such as user authentication, appointment booking, payment processing, staff management, provider analytics, notifications, and administrative operations work together seamlessly without conflicts or data inconsistencies.

After the completion of development cycles, system testing evaluates whether the overall system meets specified functional requirements, performance benchmarks, and quality standards. It focuses on end-to-end workflows to confirm that data flows smoothly across components and that user actions produce expected outcomes. Successful system testing provides confidence that the platform is stable, reliable, and ready for deployment, ensuring consistent and high-quality experience for all users.

### 5.4.2 Unit Testing

Unit testing in Salon Management System focuses on verifying each functional component independently to detect issues at the smallest level. Core units tested include user login and authentication, appointment booking logic, payment processing, staff management functions, service CRUD operations, and provider analytics. For example, tests ensure that user input (such as email, password, or booking details) meets expected formats, that appointment slots save and update correctly, and that payment modules return valid outputs when provided with valid card details. Unit tests are repeatable and are executed frequently during development to ensure that updates or code changes do not break existing functionality. This module-level testing enables developers to maintain a stable foundation before proceeding to integration, system, and acceptance testing.

**Unit Testing 1:** Authentication and User Management

**Testing Objective:** To ensure that the login, signup, password reset, and role-based authentication functionalities work correctly and securely in table 6.1.

**Table 0.1** UT 1 Authentication and User Management

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| AUTH-01 | Login with valid credentials | Email + Correct Password | Login success, | Login success, | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | | | dashboard displayed | dashboard displayed | |
| AUTH-02 | Login with invalid password | Email + Wrong Password | “Invalid credentials” error | “Invalid credentials” error | Pass |
| AUTH-03 | Signup with missing fields | Email only | Field validation errors | Field validation errors | Pass |
| AUTH-04 | Customer registration | New customer data | Account created, redirect to dashboard | Account created, redirect to dashboard | Pass |
| AUTH-06 | Provider registration | New provider data | Account pending admin approval | Account pending admin approval | Pass |
| AUTH-06 | Password reset request | Registered email | Reset link sent | Reset link sent | Pass |

**Unit Testing 2:** Appointment Booking Module

**Testing Objective:** To ensure that customers can book appointments, check availability, cancel bookings, and receive confirmations correctly as shown in table 6.2.

**Table 0.2** UT 2 Appointment Booking Module

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| BOOK-01 | Create new booking | ServiceID, StaffID, Date, Time | Booking saved successfully | Booking saved successfully | Pass |
| BOOK-02 | Check availability | Staff ID, Date, Time | Available or not available status | Available or not available status | Pass |
| BOOK-03 | Prevent double booking | Same staff, same time slot | "Time slot unavailable" error | "Time slot unavailable" error | Pass |

| | | | | | |
|---|---|---|---|---|---|
| BOOK-04 | Cancel existing booking | Booking ID | Status updated to cancelled | Status updated to cancelled | Pass |
| BOOK-05 | View customer bookings | Customer ID | List of upcoming and past bookings | List of upcoming and past bookings | Pass |
| BOOK-06 | Book with past date | Date less than today | "Invalid date selection" error | "Invalid date selection" error | Pass |

**Unit Testing 3:** Payment Processing

**Testing Objective:** To validate that Stripe payment integration works correctly for booking payments as given in table 6.3.

**Table 0.3** UT 3 Payment Processing

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| PAY-01 | Create payment intent | Booking ID, Amount | Stripe client secret returned | Stripe client secret returned | Pass |
| PAY-02 | Confirm successful payment | Valid card details | Payment successful, booking confirmed | Payment successful, booking confirmed | Pass |
| PAY-03 | Handle declined payment | Test declined card | Payment failed, error message shown | Payment failed, error message shown | Pass |
| PAY-04 | Process webhook event | Stripe webhook payload | Booking status updated | Booking status updated | Pass |
| PAY-05 | Generate invoice | Booking ID | PDF invoice generated | PDF invoice generated | Pass |

| | | | | | |
|---|---|---|---|---|---|
| PAY-06 | Handle refund | Booking ID | Payment refunded, updated | Payment refunded, updated | Pass |

**Unit Testing 4:** Staff Management Module

**Testing Objective:** To ensure that salon owners can add, edit, delete staff and manage staff schedules as provided in table 6.4.

**Table 0.4** UT 4 Staff Management Module

| **Test Case ID** | **Test Description** | **Input** | **Expected Output** | **Actual Output** | **Remarks** |
|---|---|---|---|---|---|
| STAFF-01 | Add new staff member | Name, Role, Commission | Staff saved successfully | Staff saved successfully | Pass |
| STAFF-02 | Edit staff details | Staff ID, Updated info | Staff details updated | Staff details updated | Pass |
| STAFF-03 | Delete staff member | Staff ID | Staff removed from system | Staff removed from system | Pass |
| STAFF-04 | Set staff schedule | Staff ID, Working hours | Schedule saved | Schedule saved | Pass |
| STAFF-05 | Calculate staff commission | Staff ID, Booking amount | Commission calculated correctly | Commission calculated correctly | Pass |
| STAFF-06 | View staff appointments | Staff ID, Date range | List of appointments displayed | List of appointments displayed | Pass |

**Unit Testing 5:** Service Management Module

**Testing Objective:** To verify that salon owners can manage services offered by their salon as shown in table 6.5.

**Table 0.5** UT 5 Service Management Module

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| SERV-01 | Add new service | Name, Price, Duration, Category | Service saved | Service saved | Pass |
| SERV-02 | Edit service details | Service ID, Updated price | Service updated | Service updated | Pass |
| SERV-03 | Delete service | Service ID | Service removed | Service removed | Pass |
| SERV-04 | View all services | Provider ID | List of services displayed | List of services displayed | Pass |
| SERV-05 | Assign service to staff | Service ID, Staff ID | Staff assigned to service | Staff assigned to service | Pass |
| SERV-06 | Service availability check | Service ID, Date | Available staff list returned | Available staff list returned | Pass |

**Unit Testing 6:** Provider Dashboard Module

**Testing Objective:** To ensure that salon owners can view revenue analytics, booking statistics, and business reports correctly as given in table 6.6.

**Table 0.6** UT 6 Provider Dashboard Module

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| DASH-01 | View revenue analytics | Provider ID, Date range | Total revenue displayed | Total revenue displayed | Pass |

| | | | | | |
|---|---|---|---|---|---|
| DASH-02 | View booking statistics | Provider ID | Total bookings, completed, cancelled shown | Total bookings, completed, cancelled shown | Pass |
| DASH-03 | View popular services | Provider ID | Top services by bookings | Top services by bookings | Pass |
| DASH-04 | View staff performance | Provider ID | Staff ranking by revenue | Staff ranking by revenue | Pass |
| DASH-05 | View expense tracking | Provider ID | Expense list and summary | Expense list and summary | Pass |

**Unit Testing 7:** Customer Dashboard Module

**Testing Objective:** To validate that customers can view upcoming appointments, booking history, and manage their profile as shown in table 6.7.

**Table 0.7** UT 7 Customer Dashboard Module

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| CUST-01 | View upcoming appointments | Customer ID | List of future bookings | List of future bookings | Pass |
| CUST-02 | View booking history | Customer ID | List of past bookings | List of past bookings | Pass |
| CUST-03 | Reschedule appointment | Booking ID, New time | Booking updated | Booking updated | Pass |
| CUST-04 | View payment history | Customer ID | List of transactions | List of transactions | Pass |

| | | | | | |
|---|---|---|---|---|---|
| CUST-06 | Update profile | Name, Email, Phone | Profile updated | Profile updated | Pass |

**Unit Testing 8:** Admin Module

**Testing Objective:** To ensure that admin can manage providers, users, and view system analytics as provided in table 6.8.

**Table 6.8** UT 8 Admin Module

| Test Case ID | Test Case Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| AUTH-01 | Login with valid credentials | Email + Correct Password | Login success, dashboard displayed | Login success, dashboard displayed | Pass |
| AUTH-02 | Login with invalid password | Email + Wrong Password | "Invalid credentials" error | "Invalid credentials" error | Pass |
| ADMIN-03 | Reject provider registration | Provider ID | Provider removed | Provider removed | Pass |
| ADMIN-04 | View all customers | Admin login | List of all customers | List of all customers | Pass |
| ADMIN-05 | View system analytics | Admin login | Total users, bookings, revenue | Total users, bookings, revenue | Pass |
| ADMIN-06 | Disable user account | User ID | Account disabled | Account disabled | Pass |

### 5.4.3 Functional Testing

Functional testing in the Salon Management System is carried out after unit testing to ensure that every module behaves exactly as defined in the system requirements. All major user roles

Customer, Provider, and Admin are tested to confirm that their respective features load correctly, display accurate data, and respond as expected to user actions..

**Functional Testing 1:** Login (All User Roles)

**Objective:** To ensure that all user roles Customer, Provider, and Admin can log in securely, are redirected to their respective dashboards, and proper validation and logout functionality works given in table 6.9.

**Table 0.8** FT 1 Login (All User Roles)

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| FT-001 | Login with valid Customer credentials | Email: customer @test.com Pass word: ******** | Customer dashboard loads with upcoming appointments | Customer dashboard loads with upcoming appointments | Pass |
| FT-002 | Login with valid Provider credentials | Email: provider @test.com Pass word: ******** | Provider dashboard loads with revenue analytics | Provider dashboard loads with revenue analytics | Pass |
| FT-003 | Login with valid admin credentials | Email: admin@t est.com Passwor d: ******** | Admin dashboard loads with system overview | Admin dashboard loads with system overview | Pass |
| FT-004 | Login with invalid credentials | Wrong email or wrong password | System displays “Invalid credentials” | System displays “Invalid credentials” | Pass |
| FT-005 | Login with empty fields | Email: empty Password: empty | Validation message displayed: “All fields are required” | Validation message displayed: “All | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | | | | fields are required” | |
| FT-006 | Logout | User clicks logout button | User returns to login screen and session ends | User returns to login screen and session ends | Pass |

The functional testing table for the login process evaluates the behavior of the Salon Management System authentication system across all three user categories Customers, Providers, and Admins. Test Case FT-001 verifies the customer login flow by checking whether valid credentials correctly redirect the customer to a personalized dashboard displaying their upcoming appointments and booking history. Following this, Test Case FT-002 ensures that providers logging in with valid details are accurately directed to their designated dashboard, which includes revenue analytics, booking statistics, and staff performance. Test Case FT-003 confirms that administrators gain access to the system overview dashboard upon entering correct login information. Test Case FT-004 examines the system's ability to detect invalid login attempts by providing an incorrect email or password; the expected "Invalid credentials" message is displayed, indicating proper error handling. Test Case FT-005 ensures robust validation by testing empty fields, where the system appropriately prompts the user that all fields are required. Finally, Test Case FT-006 confirms that the logout feature terminates the user session securely and redirects them back to the login interface. All test cases passed successfully, demonstrating that the Salon Management System's authentication module consistently delivers secure access control and accurate dashboard routing for all user types.

**Functional Testing 2: Customer Module**

**Objective:** To verify that customers can browse services, book appointments, make payments, view history, and manage their profile correctly as given in table 6.10.

**Table 0.9** FT 2 Customer Module

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| FT-C001 | View dashboard | Customer login | Dashboard shows upcoming appointments | Dashboard shows upcoming appointments | Pass |
| FT-C002 | Browse services | Service category list | Services displayed with price and duration | Services displayed with price and duration | Pass |
| FT-C003 | Book appointment | Service, Staff, Time slot | Booking confirmed, email sent | Booking confirmed, email sent | Pass |
| FT-C004 | Make payment | Valid card details | Payment successful, booking confirmed | Payment successful, booking confirmed | Pass |
| FT-C005 | View booking history | Navigate to My Bookings | Past and upcoming bookings shown | Past and upcoming bookings shown | Pass |
| FT-C006 | Cancel booking | Booking ID, Confirm cancel | Status changed to cancelled | Status changed to cancelled | Pass |
| FT-C007 | Update profile | Name, Email, Phone | Profile updated instantly | Profile updated instantly | Pass |
| FT-C008 | Leave review | Completed booking ID, Rating, Comment | Review saved and displayed | Review saved and displayed | Pass |

The functional testing table for the Customer Module evaluates the essential features that support a smooth booking and service experience within the Salon Management System. Test Case FT-C001 verifies that the customer dashboard loads properly after login, displaying upcoming appointments and booking history. Test Case FT-C002 checks the service browsing functionality, confirming that all services are displayed with correct prices and durations. Test Case FT-C003 validates the complete booking flow from service selection to confirmation, including email notification. Test Case FT-C004 ensures that Stripe payment processing works correctly and booking status updates after successful payment. Test Case FT-C005 verifies that customers can view their complete booking history with status of each appointment. Test Case FT-C006 checks the cancellation feature, confirming that cancelled appointments are removed from upcoming list. Test Case FT-C007 validates profile update functionality, ensuring changes are saved and reflected immediately. Finally, Test Case FT-C008 tests the review system, confirming that customers can leave ratings and comments for completed services. All tests passed successfully, indicating that the Customer Module functions reliably and supports seamless booking and service management.

**Functional Testing 3:** Provider Module

**Objective:** To verify that salon owners can manage services, staff, view analytics, track expenses, and manage subscriptions correctly as given in table 6.11..

**Table 0.10** FT 3 Provider Module

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| FT-P001 | Provider registration | Business name, address, phone | Account pending admin approval | Account pending admin approval | Pass |
| FT-P002 | View dashboard | Provider login | Revenue analytics, booking stats shown | Revenue analytics, booking stats shown | Pass |
| FT-P003 | Add new service | Name, Price, Duration, Category | Service added to list | Service added to list | Pass |
| FT-P004 | Edit service | Service ID, Updated price | Service updated | Service updated | Pass |
| FT-P005 | Auto-format portfolio | One-click formatting | Consistent layout applied | Consistent layout applied | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | | | across all sections | across all sections | |
| FT-P005 | Add new staff | Name, Role, Commission rate | Staff added to list | Staff added to list | Pass |
| FT-P006 | Set staff schedule | Staff ID, Working hours | Schedule saved | Schedule saved | Pass |
| FT-P007 | View calendar | Date range | Appointments displayed on calendar | Appointments displayed on calendar | Pass |
| FT-P008 | Track expense | Amount, Category, Description | Expense added to list | Expense added to list | Pass |
| FT-P009 | Manage subscription | Select plan, Stripe payment | Subscription activated | Subscription activated | Pass |

The functional testing table for the Provider Module evaluates features that support salon owners in managing their business operations. Test Case FT-P001 verifies that new providers can register and their accounts go to pending admin approval status. Test Case FT-P002 checks that the provider dashboard displays revenue analytics and booking statistics correctly. Test Case FT-P003 validates the service creation feature, ensuring services can be added with all required fields. Test Case FT-P004 confirms that existing services can be edited and updated. Test Case FT-P005 tests staff management, verifying that new staff members can be added to the system. Test Case FT-P006 ensures that staff working hours can be set and saved. Test Case FT-P007 validates that appointments are displayed correctly on the calendar view. Test Case FT-P008 checks the expense tracking feature, confirming that business expenses can be recorded. Finally, Test Case FT-P009 tests subscription management, verifying that providers can select and pay for subscription plans via Stripe. All tests passed successfully, confirming that the Provider Module reliably supports all salon management operatios.

**Functional Testing 4:** Admin Module

**Objective:** To ensure that admin can manage providers, users, view system analytics, and perform platform oversight as shown in table 6.12

**Table 0.11** FT 4 Admin Module

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| FT-A001 | Admin login | Special admin credentials | Admin dashboard loads | Admin dashboard loads | Pass |
| FT-A002 | View all providers | Admin login | List of all registered providers | List of all registered providers | Pass |
| FT-A003 | Approve pending provider | Provider ID, Approve button | Provider status becomes active | Provider status becomes active | Pass |
| FT-A004 | Reject pending provider | Provider ID, Reject button | Provider removed from system | Provider removed from system | Pass |
| FT-A003 | Approve pending provider | Provider ID, Approve button | Provider status becomes active | Provider status becomes active | Pass |
| FT-A004 | Reject pending provider | Provider ID, Reject button | Provider removed from system | Provider removed from system | Pass |
| FT-A007 | Disable user account | User ID | User account disabled | User account disabled | Pass |
| FT-A008 | View subscription reports | Admin login | List of active subscriptions | List of active subscriptions | Pass |

The functional testing table for the Admin Module ensures that administrative tasks can be performed correctly. Test Case FT-A001 verifies that admin login grants access to the admin dashboard. Test Case FT-A002 checks that all registered providers are displayed in a list. Test Case FT-A003 validates that pending provider registrations can be approved, changing their status to active. Test Case FT-A004 confirms that provider registrations can be rejected and removed from the system. Test Case FT-A005 ensures that all registered customers are visible to the admin. Test Case FT-A006 tests the system analytics feature, verifying that charts display correct data. Test Case FT-A007 checks that user accounts can be disabled when necessary. Finally, Test Case FT-A008 confirms that subscription reports are available for admin review. All tests passed successfully, demonstrating that the Admin Module provides complete platform oversight capabilities.

**Functional Testing 5** Automated Notifications

**Objective:** To confirm that automated email notifications are sent correctly for booking confirmations, reminders, and payment receipts as shown in table 6.13.

**Table 0.12** FT 5 Automated Notifcations

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| FT-N001 | Booking confirmation email | Complete booking payment | Email received with booking details | Email received with booking details | Pass |
| FT-N002 | Appointment reminder | 24 hours before booking time | Email reminder received | Email reminder received | Pass |
| FT-N003 | Payment receipt email | Successful payment | Email receipt with amount details | Email receipt with amount details | Pass |
| FT-N004 | New booking notification to provider | Customer books appointment | Provider receives email notification | Provider receives email notification | Pass |

| | | | | | |
|---|---|---|---|---|---|
| FT-N005 | Subscription expiry reminder | 7 days before expiry | Email reminder sent to provider | Email reminder sent to provider | Pass |
| FT-N006 | Booking cancellation notification | Customer cancels booking | Cancellation email sent | Cancellation email sent | Pass |

The functional testing of the Automated Notification System ensures that all email communications are sent reliably and on time. Test Case FT-N001 verifies that customers receive a booking confirmation email immediately after successful payment. Test Case FT-N002 checks that appointment reminder emails are sent 24 hours before the scheduled appointment time. Test Case FT-N003 confirms that payment receipts are emailed after each successful transaction. Test Case FT-N004 validates that providers receive notifications when new bookings are made for their salon. Test Case FT-N005 ensures that subscription expiry reminders are sent 7 days before the expiry date. Finally, Test Case FT-N006 confirms that customers receive cancellation emails when they cancel an appointment. All test cases passed, demonstrating that the notification system works reliably using Bull queue and Redis for background processing.

**Functional Testing 6:** Subscription Management

**Objective:** To ensure that providers can subscribe to plans, access features based on subscription, and handle expiry correctly as given in table 6.14.

Table 0.13 FT 6 Subscription Management

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| FT-S001 | View available plans | Provider login | Basic, Premium, Enterprise plans shown | Basic, Premium, Enterprise plans shown | Pass |
| FT-S002 | Subscribe to Premium plan | Select Premium, Stripe payment | Subscription active, premium | Subscription active, premium | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | | | features accessible | features accessible | |
| FT-S003 | Access premium feature | Active subscription | Analytics dashboard accessible | Analytics dashboard accessible | Pass |
| FT-S004 | Access premium feature without subscription | No active subscription | Feature blocked, upgrade prompt | Feature blocked, upgrade prompt | Pass |
| FT-S005 | Auto-expire subscription | Subscription end date passed | Status changed to expired | Status changed to expired | Pass |
| FT-S006 | Cancel subscription | Cancel button | Subscription cancelled, no further billing | Subscription cancelled, no further billing | Pass |

The functional testing of the Subscription Management module ensures that providers can subscribe to plans and access features appropriately. Test Case FT-S001 verifies that all available subscription plans are displayed correctly. Test Case FT-S002 confirms that providers can subscribe to a plan and make payment via Stripe. Test Case FT-S003 validates that premium features become accessible after subscription activation. Test Case FT-S004 ensures that features are blocked for providers without active subscriptions. Test Case FT-S005 checks that subscriptions expire automatically after the end date. Finally, Test Case FT-S006 confirms that providers can cancel their subscriptions. All tests passed, demonstrating that the subscription module functions correctly.

**Functional Testing 7:** Review and Rating System

**Objective:** To verify that customers can leave reviews and ratings for completed services as shown in table 6.15.

**Table 0.14** FT 7 Review and Rating System

| **Test Case ID** | **Test Description** | **Input** | **Expected Output** | **Actual Output** | **Remarks** |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| FT-R001 | Leave review for completed service | Booking ID, Rating (5 stars), Comment | Review saved | Review saved | Pass |
| FT-R002 | View reviews on service page | Service ID | Reviews displayed with ratings | Reviews displayed with ratings | Pass |
| FT-R003 | View reviews on staff profile | Staff ID | Reviews displayed | Reviews displayed | Pass |
| FT-R004 | Provider responds to review | Review ID, Response text | Response saved and visible | Response saved and visible | Pass |
| FT-R005 | Average rating calculation | Multiple reviews | Average displayed correctly | Average displayed correctly | Pass |
| FT-R006 | Prevent review without completed booking | Uncompleted booking | Error message, review not allowed | Error message, review not allowed | Pass |

The functional testing of the Review and Rating System ensures that customers can provide feedback and ratings for services they have received. Test Case FT-R001 verifies that customers can leave 5-star ratings and comments for completed services. Test Case FT-R002 confirms that reviews are visible on service pages for other customers to see. Test Case FT-R003 checks that staff profiles also display customer reviews. Test Case FT-R004 validates that salon owners can respond to customer reviews publicly. Test Case FT-R005 ensures that average ratings are calculated correctly based on all reviews. Finally, Test Case FT-R006 confirms that customers cannot leave reviews for appointments that were not completed. All tests passed, demonstrating that the review system works reliably.

### 5.4.4 Integration Testing

Integration testing in the Salon Management System is a critical phase that focuses on verifying the interactions between individual modules, such as the Customer Module, Provider Dashboard, payment processing system, staff management system, and admin controls. This testing ensures that these components communicate correctly, exchange data seamlessly, and work together as intended. By detecting interface inconsistencies, logical errors, and potential data mismatches, integration testing validates that the platform functions smoothly in real-world scenarios, providing reliable appointment booking, accurate payment processing, and consistent customer and provider workflows. This phase enhances the overall system reliability and prepares the Salon Management System for subsequent testing and deployment stages.

**Integration Testing 1:** Authentication and Booking Integration

**Objective:** To ensure that authentication and booking modules work together seamlessly as shown in table 6.16.

Table 0.15 IT 1 Authentication and Booking Integration

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| IT-001 | Customer login creates session | Valid customer credentials | Session created, booking page accessible | Session created, booking page accessible | Pass |
| IT-002 | Book without login | Booking attempt | Redirect to login page | Redirect to login page | Pass |
| IT-003 | JWT validation on protected routes | Valid JWT token | Access granted to booking API | Access granted to booking API | Pass |
| IT-004 | Expired token handling | Expired JWT token | Redirect to login | Redirect to login | Pass |
| IT-005 | Role-based access to booking | Provider tries customer booking API | Access denied | Access denied | Pass |

The integration testing for the booking and payment workflow ensures that critical modules such as appointment creation, Stripe payment processing, webhook handling, and invoice generation work smoothly together. Test Case IT-006 confirms that after a customer completes a successful payment, the booking status is automatically updated from pending to confirmed, ensuring correct order processing. Test Case IT-007 validates that when a payment fails or is declined, the booking remains in pending status and an appropriate error message is shown to the customer. Test Case IT-008 checks that Stripe webhook events are properly received and processed, updating payment status in the database without manual intervention. Test Case IT-009 verifies that payment requests without a valid booking ID are rejected with an error message. Finally, Test Case IT-010 confirms that after successful payment, an invoice is automatically generated and stored in the database for future reference. All integration points passed successfully, demonstrating stable connectivity across booking and payment modules Additionally, these tests confirm that the payment system handles concurrent transactions correctly, maintaining data integrity even under multiple simultaneous payment attempts. The seamless coordination between booking and Stripe payment gateway highlights the robustness of the system architecture, ensuring reliable financial operations across the platform.

**Integration Testing 2:** Booking and Payment Integration

**Objective:** To verify that booking and payment modules work together correctly as given in table 6.17.

**Table 0.16** IT 2 Booking and Payment Integration

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| IT-001 | Booking after successful payment | Complete booking, pay | Booking status = CONFIRMED | Booking status = CONFIRMED | Pass |
| IT-002 | Failed payment handling | Complete booking, declined card | Booking not confirmed, error shown | Booking not confirmed, error shown | Pass |
| IT-003 | Stripe webhook sync | Payment success webhook | Booking payment status updated | Booking payment status updated | Pass |

| | | | | | |
|---|---|---|---|---|---|
| IT-004 | Payment without booking | Invalid request | Error message | Error message | Pass |
| IT-005 | Invoice generation after payment | Successful payment | Invoice created in database | Invoice created in database | Pass |

The integration testing for the booking and payment workflow ensures that critical modules such as appointment creation, Stripe payment processing, webhook handling, and invoice generation work smoothly together. Test Case IT-006 confirms that after a customer completes a successful payment, the booking status is automatically updated from pending to confirmed, ensuring correct order processing. Test Case IT-007 validates that when a payment fails or is declined, the booking remains in pending status and an appropriate error message is shown to the customer. Test Case IT-008 checks that Stripe webhook events are properly received and processed, updating payment status in the database without manual intervention. Test Case IT-009 verifies that payment requests without a valid booking ID are rejected with an error message. Finally, Test Case IT-010 confirms that after successful payment, an invoice is automatically generated and stored in the database for future reference. All integration points passed successfully, demonstrating stable connectivity across booking and payment modules.

Additionally, these tests confirm that the payment system handles concurrent transactions correctly, maintaining data integrity even under multiple simultaneous payment attempts. The seamless coordination between booking and Stripe payment gateway highlights the robustness of the system architecture, ensuring reliable financial operations across the platform.

**Integration Testing 3:** Booking Queue and Email Integration

**Objective:** To ensure that booking, queue, and email notification modules work together as shown in table 6.18.

Table 0.17 IT 3 Booking Queue and Email Integration

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| IT-001 | Email after booking | Complete booking | Queue job created, email sent | Queue job created, email sent | Pass |

| | | | | | |
|---|---|---|---|---|---|
| IT-002 | Reminder email job | Booking for tomorrow | Cron triggers, email sent | Cron triggers, email sent | Pass |
| IT-003 | Failed email retry | Email service fails | Queue retries, email sent | Queue retries, email sent | Pass |
| IT-004 | Provider notification | New booking created | Email sent to provider | Email sent to provider | Pass |

The integration testing for the booking, queue, and email notification workflow ensures that critical modules such as appointment creation, Bull queue, Redis, and email service work smoothly together. Test Case IT-011 confirms that after a customer successfully books an appointment, a job is added to the Bull queue and an email confirmation is sent to the customer without delaying the booking response. Test Case IT-012 validates that the cron scheduler triggers the reminder job at the correct time (24 hours before appointment) and that the email reminder is successfully delivered. Test Case IT-013 checks that if the email service fails temporarily, the Bull queue retries the job automatically until successful delivery. Finally, Test Case IT-014 confirms that salon owners receive email notifications when a new booking is made for their salon, ensuring they are immediately aware of new appointments. All integration points passed successfully, demonstrating stable connectivity across booking, queue, and email notification modules.

**Integration Testing 4:** Provider Staff and Service Integration

**Objective:** To verify that provider, staff, and service management modules work together correctly as shown in table 6.19.

Table 0.18 IT 4 Provider Staff and Service Integration

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|---|
| IT-015 | Staff assigned to service | Service ID, Staff ID | Service shows staff availability | Service shows staff availability | Pass |

| | | | | | |
|---|---|---|---|---|---|
| IT-016 | Booking reflects staff schedule | Book with staff off day | Staff not available for that time | Staff not available for that time | Pass |
| IT-017 | Commission calculation | Completed booking | Staff commission recorded | Staff commission recorded | Pass |
| IT-018 | Service deletion affects future bookings | Delete service | Customers cannot book deleted service | Customers cannot book deleted service | Pass |

The integration testing for the provider, staff, and service management workflow ensures that critical modules such as provider dashboard, staff management, service management, and booking logic work smoothly together. Test Case IT-015 confirms that when a provider assigns a staff member to a specific service, the system correctly links them and displays that staff member as available for that service during customer booking. Test Case IT-016 validates that if a customer tries to book an appointment with a staff member who is marked as off on that day, the system correctly shows the staff as unavailable and prevents the booking. Test Case IT-017 checks that after a booking is marked as completed, the staff member's commission is automatically calculated and recorded in the system for payroll purposes. Finally, Test Case IT-018 confirms that when a provider deletes a service, customers can no longer book that service for future appointments, and existing bookings remain unchanged. All integration points passed successfully, demonstrating stable connectivity across provider, staff, and service management modules.

Additionally, these tests confirm that the system handles concurrent updates to staff schedules and service assignments efficiently, maintaining data integrity under multiple simultaneous operations. The seamless coordination between provider dashboard, staff schedules, and service availability highlights the robustness of the system architecture, ensuring reliable business operations across the platform.

## 5.5 Tools and Technologies

Table 6.22 provides a comprehensive overview of the key tools and technologies utilized throughout the development of the Salon Management System project, clearly outlining their respective versions and specific purposes. The table includes software tools used for documentation, presentation, and system design such as MS Word and PowerPoint for formal reporting and presentations.

In addition, the table highlights backend development and data handling technologies, including Node.js and Express.js for implementing server-side logic, along with PostgreSQL as the primary database with Prisma ORM for efficient data storage and retrieval. Redis and Bull Queue are used for caching and background job processing. Stripe Payment Gateway is integrated for secure payment processing.

Furthermore, the table lists tools for API testing and version control, including Postman for validating backend endpoints and Git with GitHub for source code management, collaboration, and version tracking. Finally, frontend technologies such as HTML, CSS, JavaScript, Bootstrap 5, and React.js are included, demonstrating their contribution to building responsive, accessible, and user-friendly interfaces. Collectively, these tools and technologies ensure a structured, scalable, and efficient development workflow, support maintainability and future enhancements, and enable smooth collaboration throughout the software development lifecycle.

**Table 0.19** Tools and Technologies

| | Tools | Version | Rationale |
|---|---|---|---|
| | MS Word | 2019 | Documentation and report writing |
| | MS PowerPoint | 2019 | Presentation creation for project defense |
| | Adobe Illustrator | 2021 | UI design and graphics creation |
| | Figma | Latest | UI/UX design and prototyping |

| | | | |
|---|---|---|---|
| **Tools And Technologies** | | | |
| | Postman | Latest | API testing and integration |
| | Git and GitHub | Latest | Version control and collaborative development |
| | MS Project | 2019 | Gant Chart |
| | Node.js | 20.x | Primary backend runtime |
| | Express.js | 4.x | Web framework for REST APIs |
| | PostgreSQL | 15+ | Primary relational database |
| | Prisma | 5.x | Database ORM for type-safe queries |
| | Redis | Latest | In-memory cache and queue broker |
| | Bull | Latest | Background job processing |
| | Stripe | Latest | Online payment processing |
| | React.js, Bootstrap 5, CSS3 | 18.x | Frontend UI framework |
| | Axios | Latest | API calls from frontend |
| | node-cron | Latest | Scheduled cron jobs |

## 5.6 Summary

This chapter presents the various testing approaches applied to the Salon Management System platform to ensure system reliability, correctness, and overall quality. The testing strategy includes unit testing, functional testing, and integration testing, each targeting different levels of system validation. All major modules such as user authentication and login, appointment booking, payment processing, staff and service management, provider dashboards, customer features, and admin controls were systematically evaluated using well-structured test cases.

These test cases were designed to cover both standard operational scenarios and critical edge-case conditions to verify robustness under diverse user behaviors and data inputs. Through this comprehensive testing process, the platform's functional accuracy, module interoperability, and user experience were thoroughly assessed, ensuring that the system meets defined requirements and performs consistently across all use cases.

# Chapter 7
# Conclusion and Future Work

# 6 Conclusion and Future Work

## 6.1 Conclusion

The Salon Management System is designed to streamline the online booking, staff management, payment processing, and business analytics operations for modern salons, while offering efficient tools to customers, salon owners, and administrators. The primary objective of this project was to develop a comprehensive digital ecosystem where salon owners can manage their business operations, track revenue, and receive automated insights, all within a single platform. For customers, the system provides convenient tools to book appointments, make secure payments, and view service history, while salon owners gain complete business visibility and actionable suggestions for operational improvement.

The Salon Management System has successfully transformed the salon management process, making it more organized, efficient, and user-friendly. Traditionally, salon owners relied on paper registers, scattered WhatsApp messages, and manual Excel sheets, which were often inefficient and prone to errors. With this system, all relevant business information including customer records, staff schedules, booking data, and payment history is centralized, digitally accessible, and easily manageable through an intuitive dashboard.

A significant advantage of the Salon Management System is its intuitive and responsive interface. Users do not need technical expertise to navigate the platform. Whether managing appointments, viewing staff schedules, processing payments, or checking business analytics, customers and salon owners can interact with the system effortlessly. The platform ensures seamless navigation, clear workflows, and a smooth user experience across all modules including booking, payment, staff management, and analytics.

During development and testing, rigorous quality assurance was performed on features including user authentication, appointment booking, payment processing, staff and service management, provider dashboards, automated notifications, and subscription management. All modules have been thoroughly tested and operated as intended, ensuring reliability and consistency.

In conclusion, the Salon Management System effectively achieves its purpose of enhancing salon operations while delivering efficient, automated business management tools. It consolidates scattered information, simplifies appointment booking and staff management, and increases visibility for business performance. By combining user-friendly design, automated notifications, and robust management tools, the system offers a complete solution that is both practical and forward-looking. It represents a significant advancement in integrating technology into salon business management, making operations more accessible, efficient, and impactful for all stakeholders. The platform addresses existing challenges in salon management while laying the groundwork for future enhancements, including mobile applications, SMS notifications, inventory management, and loyalty programs, further cementing its role in the salon management landscape.

## 6.2 Future Work

Although the current version of the Salon Management System is fully functional and successfully meets its objectives, there is substantial scope to enhance its features, usability, accessibility, and overall user experience. The future development of the Salon Management System aims to make the platform smarter, more interactive, and inclusive, while ensuring data security and seamless performance. The following initiatives are proposed for future work:

### 6.2.1 Mobile Applications (Android and iOS)

Developing dedicated mobile applications for both Android and iOS platforms will allow customers and salon owners to access the Salon Management System anytime and anywhere. The mobile apps will provide a fully responsive interface for appointment booking, service browsing, payment processing, and real-time notifications for booking confirmations and reminders. Offline features, such as viewing upcoming appointments and cached service listings, can also be integrated to enhance accessibility in areas with limited internet connectivity. Salon owners will be able to manage staff schedules and view business analytics directly from their mobile devices.

### 6.2.2 Local Payment Method Integration

To improve payment accessibility, the system will integrate additional local payment methods beyond Stripe. This includes JazzCash, EasyPaisa, bank transfers, and cash on delivery options for customers who prefer not to pay online. Partial payment and deposit options will be available for high-value services. Automated payment reconciliation and settlement reports will help salon owners track payments across multiple channels, making the platform more suitable for the Pakistani market.

### 6.2.3 Walk-in Customer Management

Future enhancements will optimize the system for walk-in customers who visit the salon without prior online booking. Features will include quick check-in using phone number or name, real-time staff availability display for immediate service assignment, and digital payment processing at the counter. Walk-in customer profiles will be automatically created and linked to booking history for future visits, ensuring seamless integration between online and offline customer experiences.

### 6.2.4 Multi-Language Support

Introducing multi-language support will make the platform accessible to a broader audience. In addition to English, languages such as Urdu and other regional languages will be supported, allowing customers and salon owners from diverse backgrounds to navigate the platform comfortably. This will also enhance inclusivity for users in multilingual contexts, increasing adoption and engagement across different regions of Pakistan.

### 6.2.5 Loyalty and Reward Program

Introducing a loyalty program will help salon owners retain existing customers and attract new ones. Features will include points accumulation for each booking, reward redemption against future services, and tiered membership levels (Silver, Gold, Platinum) with exclusive benefits. Customers will be able to track their points and rewards from their dashboard. The system will also support birthday discounts, referral bonuses, and seasonal promotion campaigns to increase customer engagement.

# 7 References


[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[2] M. Dezhkam, S. Xue, and Z. Liu, "Project Management System, Concepts and Approach Foundations," in *IOP Conference Series: Earth and Environmental Science*, vol. 310, no. 5, 2019.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," arXiv, Oct. 2019.

[4] H. Niu, S. Li, and J. Li, "MetaTrader: A reinforcement learning approach integrating diverse policies for portfolio optimization," arXiv, 2022.

[5] DeepMind, "AlphaEvolve: Google DeepMind's AI agent dreams up algorithms beyond human expertise," *Wired*, May 14, 2025.

[6] E. Stevens, "The Complete Guide to UI/UX Design," AND Academy, Jan. 16, 2023. Available: https://www.andacademy.com/resources/blog/ui-ux-design/what-is-ui-ux-design-the-complete-guide/

[7] Harvard University, "Create a Strong Resume – Mignone Center for Career Success," 2024.
Available: https://careerservices.fas.harvard.edu/resources/create-a-strong-resume/

[8] Indeed, "10 Resume Writing Tips To Help You Land a Position," 2024. Available: https://www.indeed.com/career-advice/resumes-cover-letters/10-resume-writing-tips

[9] University of Wisconsin, "Resume Writing Tips – The Writing Center," 2024. Available: https://writing.wisc.edu/handbook/resume/

[10] UX Design Institute, "A Complete Guide to the UI Design Process," Oct. 12, 2023. Available: https://www.uxdesigninstitute.com/blog/guide-to-the-ui-design-process/

[11] Columbia University, "Resumes with Impact: Creating Strong Bullet Points," 2024. Available: https://www.careereducation.columbia.edu/resources/resumes-impact-creating-strong-bullet-points

[12] Y. Benabderrezak, "UML Use Case Diagram in Simple Words," University of Boumerdes, ResearchGate, 2024.

[13] O. Ņikiforova, "Definition of a Set of Use Case Patterns for Application Systems: A Prototype-Supported Development Approach," *Applied Computer Systems*, vol. 29, no. 1, pp. 59–67, 2024.

[14] M. N. Arifin and D. Siahaan, "Structural and semantic similarity measurement of UML use case diagram," *Lontar Komputer: Jurnal Ilmiah Teknologi Informasi*, vol. 11, no. 2, pp. 88–100, 2020.

[15] N. M. Minhas, A. M. Qazi, S. Shahzadi, and S. Ghafoor, "An integration of UML sequence diagram with formal specification methods — A formal solution based on Z," *Journal of Software Engineering and Applications*, vol. 8, no. 8, pp. 372–383, 2015.

[16] S. Al-Fedaghi, "UML Sequence Diagram: An Alternative Model," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 5, pp. 635–645, 2021.

[17] TechRadar, "What are Transformer Models?," May 17, 2025. Available: https://www.techradar.com/pro/what-are-transformer-models

[18] IBM, "What Is NLP (Natural Language Processing)?," 2024. Available: https://www.ibm.com/think/topics/natural-language-processing

[19] GeeksforGeeks, "Top 7 Applications of NLP (Natural Language Processing)," Jul. 15, 2025.
Available: https://www.geeksforgeeks.org/top-7-applications-of-natural-language-processing/

[20] spaCy, "Industrial-strength Natural Language Processing in Python." Available: https://spacy.io/

[21] D. Vohra, "Implementing JSON Web Tokens (JWT) for Secure Authentication," *International Journal of Computer Applications*, vol. 179, no. 23, pp. 8–12, 2018.

[22] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1204–1215, 2016.

[23] Z. Aktaş and E. Y. Doğa Serdaroğlu, "An Introduction to Software Testing Methodologies," *Gazi University Journal of Science Part A: Engineering and Innovation*, vol. 8, no. 1, pp. 1–15, 2021.

[24] M. S. Patil, "The Overview of Software Testing: Types, Methods, and Levels," *Journal of Software Engineering Tools and Technology Trends*, 2023.

[25] N. Radziwill and G. Freeman, "Reframing the Test Pyramid for Digitally Transformed Organizations," arXiv preprint, 2020.